



# GamepadOS

Your phone is the controller

Wi-Fi or USB

Gyro steering

2.5 ms wired

Free

---

# GamepadOS

## An Interdisciplinary Analysis

The branches of study hidden inside every part of the project —  
mathematics, physics, engineering, cryptography, and the human sciences.

Mathematics

Physics

Cryptography

Signal & Control

Distributed Systems

HCI

A companion to GamepadOS — The Complete Guide.

# Contents

|                                                                             |           |
|-----------------------------------------------------------------------------|-----------|
| <b>Introduction</b>                                                         | <b>5</b>  |
| What This Analysis Does                                                     | 6         |
| One Project, Ten Disciplines                                                | 7         |
| The Map: Which Part Draws On What                                           | 7         |
| <b>Mathematics</b>                                                          | <b>9</b>  |
| The Discipline, and Where It Hides                                          | 10        |
| Boolean Algebra & Discrete Math — the Button Bitmask                        | 10        |
| Linear Algebra & Affine Maps — Bytes to Axes                                | 11        |
| Geometry & Trigonometry — Sticks and Tilt                                   | 12        |
| Numerical Methods — Curves and Interpolation                                | 12        |
| Where Each Branch Lives                                                     | 13        |
| <b>Physics</b>                                                              | <b>14</b> |
| What physics is, and the four rooms of it your project walks through        | 15        |
| Classical mechanics and inertial sensing: the phone knows which way is down | 15        |
| Electromagnetism and radio: steering across an invisible wave               | 18        |
| Electrical signalling over the wire                                         | 19        |
| Thermodynamics: heat, throttling, and the seal-gate win                     | 20        |
| <b>Electrical &amp; Computer Engineering</b>                                | <b>22</b> |
| The Discipline, and Where It Hides                                          | 23        |
| Computer Architecture — One Program, Three Instruction Sets                 | 23        |
| Memory Layout — Bytes Where You Put Them                                    | 24        |
| Real-Time Systems — Being Right and On Time                                 | 25        |
| Signalling for Priority — TOS and WMM                                       | 26        |
| Where Each Branch Lives                                                     | 26        |
| <b>Signal Processing &amp; Control Theory</b>                               | <b>27</b> |
| The Discipline, and Where It Hides                                          | 28        |
| Sampling Theory — Turning Motion Into a Stream                              | 28        |
| Digital Filtering — the One-Euro Low-Pass                                   | 29        |
| Control Shaping — Deadzones and Response Curves                             | 30        |
| Feedback Loops — Closing the Circuit                                        | 30        |
| Where Each Branch Lives                                                     | 30        |
| <b>Networking &amp; Distributed Systems</b>                                 | <b>32</b> |
| The Discipline, and Where It Hides                                          | 33        |

|                                                                                      |           |
|--------------------------------------------------------------------------------------|-----------|
| Addresses, Ports, and Subnets — Finding the Other Machine . . . . .                  | 33        |
| UDP vs TCP — Choosing to Be Unreliable . . . . .                                     | 34        |
| Discovery, Lock-On, and Failure Detection . . . . .                                  | 34        |
| Versioning as a Consistency Problem . . . . .                                        | 35        |
| Where Each Branch Lives . . . . .                                                    | 35        |
| <b>Cryptography &amp; Security . . . . .</b>                                         | <b>37</b> |
| The mathematics underneath: number theory and the prime field . . . . .              | 38        |
| Abstract algebra in the tag: the Galois field $GF(2^{128})$ . . . . .                | 40        |
| Probability and one-way functions: SHA-256 and the birthday bound . . . . .          | 40        |
| Applied cryptography: AEAD, nonces, and the reuse catastrophe . . . . .              | 41        |
| Key derivation and the QR binding that kills MITM . . . . .                          | 43        |
| Security engineering: threat model, replay window, and the hole GRX closed . . . . . | 45        |
| <b>Operating Systems &amp; Systems Programming . . . . .</b>                         | <b>47</b> |
| The Discipline, and Where It Hides . . . . .                                         | 48        |
| Concurrency — Threads, Atomics, and Races . . . . .                                  | 48        |
| The Kernel/Driver Boundary — Faking Hardware . . . . .                               | 49        |
| Processes, File Descriptors, and Packaging . . . . .                                 | 50        |
| Language Runtimes and the JNI Boundary . . . . .                                     | 50        |
| Where Each Branch Lives . . . . .                                                    | 51        |
| <b>Information &amp; Coding Theory . . . . .</b>                                     | <b>52</b> |
| Encoding and Compactness — your 20-byte frame is source coding . . . . .             | 53        |
| Entropy and Keys — measuring unpredictability . . . . .                              | 55        |
| Error Detection — catching a corrupt message . . . . .                               | 57        |
| Error Correction — surviving damage, not just spotting it . . . . .                  | 58        |
| The road not taken — deliberately no compression . . . . .                           | 59        |
| <b>Human-Computer Interaction &amp; Perception . . . . .</b>                         | <b>60</b> |
| The disciplines, named from scratch . . . . .                                        | 61        |
| Latency perception and the psychophysics of milliseconds . . . . .                   | 62        |
| Haptics and multimodal feedback . . . . .                                            | 63        |
| Input ergonomics and embodiment . . . . .                                            | 65        |
| Visual and interaction design . . . . .                                              | 66        |
| <b>Software Engineering, Databases &amp; Operations Research . . . . .</b>           | <b>68</b> |
| Software Architecture — Separation of Concerns Across Layers . . . . .               | 69        |
| Coupling, Cohesion, and the Deliberate App.tsx Monolith . . . . .                    | 70        |
| The Hybrid-App Pattern, Reused . . . . .                                             | 71        |

|                                                                            |    |
|----------------------------------------------------------------------------|----|
| Relational Database Theory — Models as Tables . . . . .                    | 71 |
| Normalization and the Set-Theoretic Uniqueness Invariant . . . . .         | 72 |
| Software-Engineering Process — Versioning, Signing, Rollback . . . . .     | 74 |
| Operations Research — The Ticket Router as an Assignment Problem . . . . . | 76 |

---

FIELD PRIMER

# Introduction

the hidden curriculum of a game controller





Figure I.1 — One project, ten disciplines.

## What This Analysis Does

You set out to build one thing: a phone that acts as a game controller for a PC. You did not set out to do mathematics, or physics, or cryptography. Yet to make that one thing work — encrypted, low-latency, on real hardware — you ended up quietly implementing a slice of nearly a dozen academic disciplines. This document is a tour of that hidden curriculum. It takes ten branches of study, one per section, and for each one shows the exact place in *your own code* where that field is doing real work.

The organizing question is simple: **for each branch of study, where does GamepadOS embody it, and why?** Every claim points at a specific file, function, constant, or byte — this is an analysis of the real system, not a general lecture. Where a discipline appears only lightly, the section says so; the goal is an honest map, not a flattering one.

### IN PLAIN TERMS

Think of your project as a house you built with your own hands. This document is a surveyor walking through it, pointing: "that load-bearing wall is structural engineering, those pipes are fluid dynamics, that wiring is electrical code." You didn't study those fields to build the house — but the house obeys them anyway, and seeing which is which makes you understand your own house far more deeply.

# One Project, Ten Disciplines

Here are the ten fields this analysis covers. They span the four great families of knowledge — the formal sciences (mathematics, computation), the natural sciences (physics), the engineering sciences (electrical, signal, network, systems), and the human sciences (perception, design, and the softer parts of software practice).

- **Mathematics** — the translations between the physical and the digital: bits, vectors, angles, curves.
- **Physics** — gravity sensing, radio waves, and the heat budget of a small hot device.
- **Electrical & Computer Engineering** — instruction sets, memory layout, and real-time scheduling.
- **Signal Processing & Control Theory** — sampling, adaptive filtering, and shaping input into feel.
- **Networking & Distributed Systems** — addresses, an unreliable transport chosen on purpose, and consistency.
- **Cryptography & Security** — elliptic curves, finite fields, and a protocol that survives an attacker on your Wi-Fi.
- **Operating Systems & Systems Programming** — threads, a kernel driver faking hardware, and file descriptors.
- **Information & Coding Theory** — compact encodings, entropy in keys, and error-correcting QR codes.
- **Human-Computer Interaction & Perception** — why milliseconds matter, and how haptics close the loop.
- **Software Engineering, Databases & Operations Research** — architecture, relational data, and a scheduling optimisation.

## NOTE

Some fits are deeper than others, and it's worth being upfront. The most *deeply* embodied disciplines are **cryptography** (a full, adversarially-verified protocol), **signal processing & control** (a real adaptive filter and shaped transfer curves), **networking & distributed systems** (transport choice, discovery, consistency), and **real-time computer engineering** (the scheduled send loop). The *lighter* touches are **physics** (mostly consuming what the sensors and radio provide, rather than deriving it) and parts of **information theory** (present in the encoding and checksums, but not a coding-theory research effort). Every section is honest about which it is.

## The Map: Which Part Draws On What

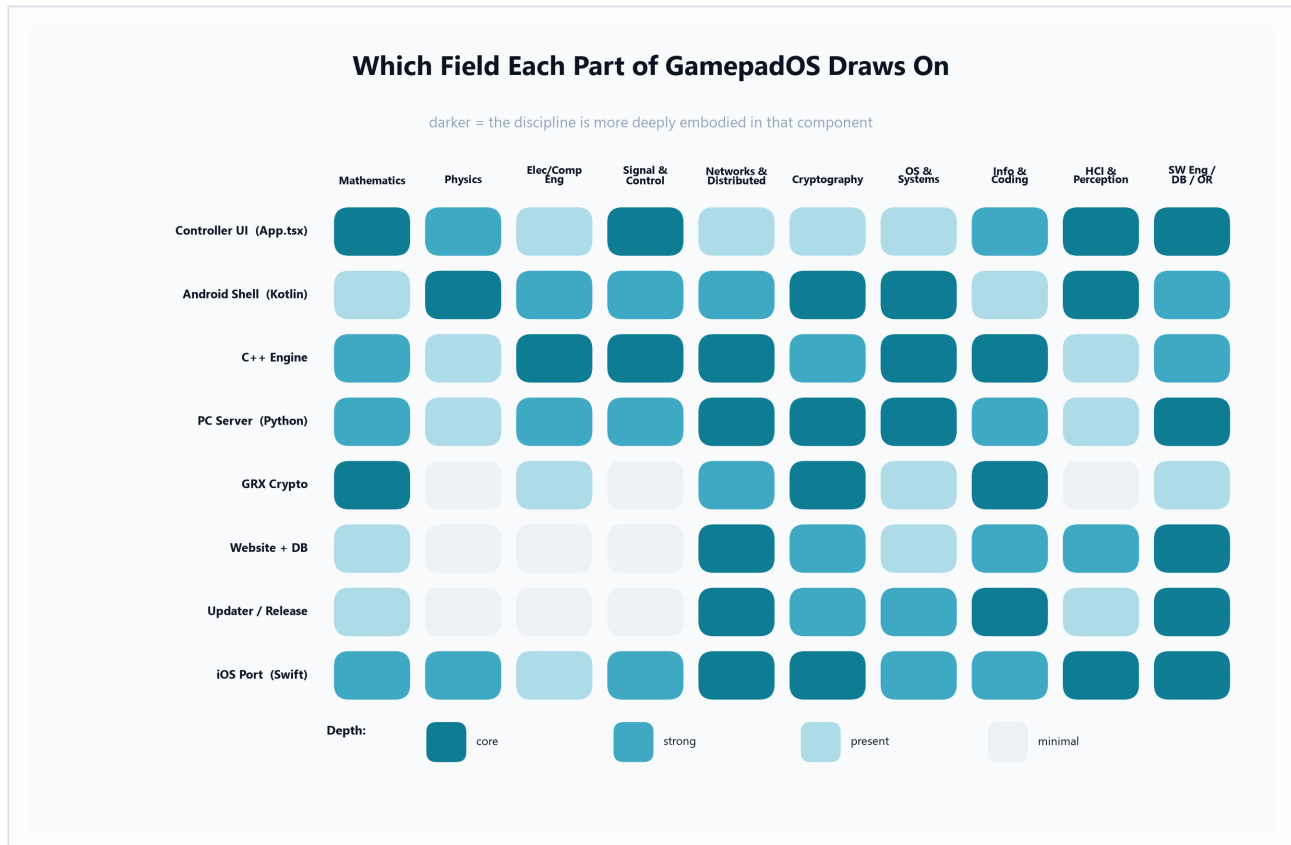


Figure I.2 — Which part of GamepadOS draws on which field (darker = deeper).

The other way to read the project is by component: each part of the system leans on a different mix of fields. The C++ engine is where real-time engineering, signal processing, and low-level networking concentrate; the GRX layer is almost pure cryptographic mathematics; the website is databases, distributed consistency, and a dash of operations research; the controller UI is geometry, control shaping, and human-computer interaction. The matrix below reads the whole project through that lens — darker cells mean the discipline runs deeper in that component.

Read the ten sections in any order; each stands alone. But read together, they make one point: a "simple" phone-as-gamepad app is, underneath, a working cross-section of the sciences and engineering disciplines it takes to move a human intention across a room and into a game — correctly, securely, and fast.



---

DISCIPLINE 1

# Mathematics

1

## The Discipline, and Where It Hides

Mathematics is the study of quantity, structure, space, and change — and it is the single field GamepadOS leans on most, usually without announcing itself. This section walks the branches of *non-cryptographic* mathematics the project embodies (the number theory behind GRX gets its own section) and points at the exact lines where each one lives.

### DEFINITION

**Discrete mathematics** — the mathematics of distinct, separate values — integers, sets, logic, graphs — as opposed to continuous quantities. Buttons that are either pressed or not, bits that are 0 or 1, and counters that step by whole numbers are all discrete math.

### DEFINITION

**Linear algebra** — the mathematics of vectors (arrows with direction and length) and linear transformations (scaling, rotating, projecting). Stick positions and coordinate mappings are linear algebra.

### DEFINITION

**Trigonometry** — the mathematics of angles and the ratios of a triangle's sides (sine, cosine, tangent, and their inverses). Turning a phone's tilt into a steering angle is trigonometry.

### IN PLAIN TERMS

You didn't sit down to "do maths." But to make a thumb-drag become a stick position, a tilt become a turn, and a button become a bit, you had to translate the physical world into numbers and back — and every one of those translations is a small theorem quietly doing its job.

## Boolean Algebra & Discrete Math — the Button Bitmask

The most pervasive math in the project is **Boolean algebra**: the algebra of true/false, invented by George Boole in 1847. Your fifteen gamepad buttons are packed into one 16-bit integer, and each button is one *bit* — one Boolean. The PC server reads them back out with bitwise operations:

apps/pc-server/server.py · lines 986-1000

Python

```

1      XB = vg.XUSB_BUTTON
2      if buttons & (1 << 0): gamepad.press_button(button=XB.XUSB_GAMEPAD_A)
3      if buttons & (1 << 1): gamepad.press_button(button=XB.XUSB_GAMEPAD_B)
4      if buttons & (1 << 2): gamepad.press_button(button=XB.XUSB_GAMEPAD_X)
5      if buttons & (1 << 3): gamepad.press_button(button=XB.XUSB_GAMEPAD_Y)
6      if buttons & (1 << 4): gamepad.press_button(button=XB.XUSB_GAMEPAD_LEFT_SHOULDER)
7      if buttons & (1 << 5): gamepad.press_button(button=XB.XUSB_GAMEPAD_RIGHT_SHOULDER)

```

```
apps/pc-server/server.py · (cont.) Python
8         if buttons & (1 << 6): gamepad.press_button(button=XB.XUSB_GAMEPAD_START)
9         if buttons & (1 << 7): gamepad.press_button(button=XB.XUSB_GAMEPAD_BACK)
10        if buttons & (1 << 8): gamepad.press_button(button=XB.XUSB_GAMEPAD_LEFT_THUMB)
11        if buttons & (1 << 9): gamepad.press_button(button=XB.XUSB_GAMEPAD_RIGHT_THUMB)
12        if buttons & (1 << 10): gamepad.press_button(button=XB.XUSB_GAMEPAD_DPAD_UP)
13        if buttons & (1 << 11): gamepad.press_button(button=XB.XUSB_GAMEPAD_DPAD_DOWN)
14        if buttons & (1 << 12): gamepad.press_button(button=XB.XUSB_GAMEPAD_DPAD_LEFT)
15        if buttons & (1 << 13): gamepad.press_button(button=XB.XUSB_GAMEPAD_DPAD_RIGHT)
```

Two mathematical ideas are doing all the work here.  $1 \ll n$  is the number  $2^n$  — shifting a single 1 bit left by  $n$  places, which is **multiplication by a power of two** expressed as a shift. `buttons & (1 << n)` is the Boolean **AND** operator, which is non-zero only if bit  $n$  is set. So this whole block is asking, fifteen times, "is this particular bit true?" That is Boolean algebra realized directly in silicon — and it's why a controller state that could have been a bulky list of names is instead two bytes.

#### DEFINITION

**Bit / bitmask** — a bit is a single binary digit (0 or 1). A bitmask is an integer used as a row of independent bits, each carrying its own true/false fact. Setting is `x |= (1<<n)` (OR); testing is `x & (1<<n)` (AND).

#### EXPERT DETAIL

This is also **set theory** in disguise. The pressed buttons form a *set*; the bitmask is that set's *characteristic function* (bit  $n$  = "is element  $n$  a member?"). Union of two button states is bitwise OR, intersection is AND — the exact correspondence between Boolean algebra and the algebra of sets that Boole and later Stone made precise.

## Linear Algebra & Affine Maps — Bytes to Axes

Every analog value in the project is a number squeezed into one byte (0–255) on the wire, then stretched back to a usable range on the other side. That stretch is an **affine map** — a straight line of the form  $y = m \cdot x + c$ . The server turns a stick byte into a joystick float:

```
apps/pc-server/server.py · lines 981-984 Python
1     def _axis(b):
2         return max(-1.0, min(1.0, (b - 128) / 127.0))
3     gamepad.left_joystick_float(x_value_float=_axis(ls_x), y_value_float=-_axis(ls_y))
4     gamepad.right_joystick_float(x_value_float=_axis(rs_x), y_value_float=-_axis(rs_y))
```

`(b - 128) / 127.0` is the affine line that maps the byte range 0...255 onto -1.0...+1.0: subtract the center (128), then scale by  $1/127$ . The `max(-1, min(1, ...))` is a **clamp**, keeping the result inside the valid interval. The leading minus on the Y axis is a **reflection** — a linear transformation that flips the axis, because screen-Y grows downward while a joystick's Y grows upward. Affine maps and reflections are the bread-and-butter of linear algebra, and here they're the glue between two number systems.

#### NOTE

The same affine thinking sets up the whole coordinate world of the UI: the controller canvas is a fixed 1263×540 pixel space, and rendering a widget is mapping its stored (x, y) into that space — a change of coordinates, which is linear algebra's home turf.

## Geometry & Trigonometry — Sticks and Tilt

A thumbstick is a **2D vector**. When you drag it, the code computes the vector's length with the **Pythagorean theorem** and applies a *circular* deadzone — geometry you can see:

```
apps/controller-ui/src/app/App.tsx · lines 924-935 TypeScript · React

1  const processStick = (x: number, y: number) => {
2    const distance = Math.sqrt(x * x + y * y);
3    const deadzoneRadius = (8 / 100) * 60; // 8% default deadzone
4    if (distance >= deadzoneRadius && distance > 0) {
5      const factor = (distance - deadzoneRadius) / (60 - deadzoneRadius);
6      return {
7        x: (x / distance) * factor * 60,
8        y: (y / distance) * factor * 60
9      };
10   }
11   return { x: 0, y: 0 };
12 }
```

`Math.sqrt(x*x + y*y)` is Pythagoras — the length of the (x, y) vector. Comparing it to `deadzoneRadius` makes the dead area a **circle**, not a square, so diagonal input behaves the same as straight input (a square deadzone would let corners drift). Then `(x / distance)` and `(y / distance)` are the vector's **unit direction** (dividing by its length gives a length-1 vector), and `factor` re-scales the magnitude so the stick ramps smoothly from the deadzone edge to full. That's **vector normalization** followed by an affine remap of magnitude — pure analytic geometry.

The gyroscope is where **trigonometry** appears. To turn "how is the phone tilted?" into steering, the Android shell reads the device's up-vector and takes inverse trig functions:

```
apps/android-client/app/src/main/java/com/gamepad/client/MainActivity.kt · lines 1265-1267 Kotlin

1  val rollDeg = STEER_SIGN * Math.toDegrees(Math.atan2(ux.toDouble(), uy.toDouble()))
2  // PITCH = how far up tips OUT of the screen plane = forward/back lean (3D look Y).
3  val pitchDeg = PITCH_SIGN * Math.toDegrees(Math.asin(uz.toDouble()))
```

`atan2(ux, uy)` recovers the **roll angle** — the direction the up-vector points *within* the screen plane — and `asin(uz)` recovers the **pitch** — how far up tilts *out* of that plane. `atan2` is the "smart" arctangent that knows all four quadrants, so a full 360° of lean is unambiguous. `Math.toDegrees` converts from radians (the natural unit of angle) to degrees. This is trigonometry doing exactly what it was invented for: recovering an angle from the ratios of a triangle's sides — here, the sides of the gravity vector.

## Numerical Methods — Curves and Interpolation

Finally, a lighter touch of **numerical methods**: shaping raw input through smooth functions. The gyro steering runs its normalized value through an **expo (cubic) response curve**:

teaching example

TypeScript · React

```
1 // blend 70% linear + 30% cubic → fine control near center, fast at the edges
2 const expo = (t) => Math.sign(t) * (0.7 * Math.abs(t) + 0.3 * Math.abs(t) ** 3);
```

This is **polynomial interpolation** used as a control curve: near the center the cubic term is tiny so steering is gentle and precise; near the edges the cube dominates so it turns fast. Blending a linear and a cubic term to trade precision against speed is a classic bit of applied numerical shaping — and the real one lives in `useGyro` in `App.tsx`.

## Where Each Branch Lives

| Branch of mathematics                                   | Where in GamepadOS                                               |
|---------------------------------------------------------|------------------------------------------------------------------|
| Boolean algebra / set theory                            | the 16-bit button bitmask ( <code>server.py</code> , the engine) |
| Number bases (binary, hex)                              | <code>1 &lt;&lt; n</code> , hex constants, endianness            |
| Linear algebra / affine maps                            | byte↔axis <code>(b-128)/127</code> , the 1263×540 canvas         |
| Analytic geometry (Pythagoras, vectors)                 | <code>processStick</code> circular deadzone + normalization      |
| Trigonometry ( <code>atan2</code> , <code>asin</code> ) | gyro roll/pitch from the up-vector                               |
| Numerical methods (interpolation)                       | the expo response curve, smoothing                               |

The pattern is consistent: wherever the physical world (a drag, a tilt, a press) meets the digital world (a byte, a bit, a float), a piece of mathematics is performing the translation. GamepadOS is, quietly, an applied-maths project wearing a game controller's clothes.

---

DISCIPLINE 2

# Physics

2

## What physics is, and the four rooms of it your project walks through

Physics is the study of matter, energy, motion, and the forces that connect them — the rulebook the physical world runs on. It is not one subject but a cluster of them, and GamepadOS, almost by accident, wanders through four of its major rooms. Every time your phone senses a tilt, every time a packet crosses the air, every time a byte travels down a USB cable, and every time the phone warms up in your hand, a different branch of physics is doing the work. This section names those branches from scratch and then points at the exact line of your own code where each one lives.

### DEFINITION

**Physics** — the science of how matter and energy behave — motion, forces, waves, electricity, heat. Its sub-branches each cover one slice of that world (motion, electromagnetism, heat, and so on).

### DEFINITION

**Classical mechanics** — the physics of everyday-scale motion and forces — how objects move, rotate, and respond to gravity. It is the branch that governs a tilting phone.

### DEFINITION

**Electromagnetism** — the physics of electric and magnetic fields, and of the waves (radio, light) they make together. It governs your Wi-Fi link.

### DEFINITION

**Thermodynamics** — the physics of heat, temperature, and energy flow. It governs why your phone gets warm and throttles.

| Sub-branch of physics                  | Where GamepadOS embodies it                                                       |
|----------------------------------------|-----------------------------------------------------------------------------------|
| Classical mechanics / inertial sensing | <code>MainActivity.kt</code> gyro fusion, <code>up</code> -vector steering math   |
| Electromagnetism / radio               | <code>getWifiInfoJson</code> band/RSSI, <code>LatencyCard</code> UI, 2.4 vs 5 GHz |
| Electrical signalling                  | <code>aoa_transport.py</code> USB bulk endpoints, RNDIS tether                    |
| Thermodynamics                         | thermal status UI, affinity removal + GRX seal-gate                               |

## Classical mechanics and inertial sensing: the phone knows which way is down

When you tilt the phone to steer, no part of the phone "sees" the tilt. Instead, two tiny sensors measure the physics of the situation and the software reconstructs the angle. This is *inertial sensing*, and it rests on two of the oldest ideas in mechanics: gravity and rotation.

**DEFINITION**

**Inertial sensing** — measuring an object's motion and orientation from forces felt inside it (acceleration, rotation) rather than from outside references like GPS or a camera.

**DEFINITION**

**Accelerometer** — a chip that measures acceleration, including the constant downward pull of gravity. Held still, it reports a vector pointing *up*, because it feels the ground pushing back against gravity.

**DEFINITION**

**Gyroscope** — a chip that measures how fast the device is *rotating* about each axis. It is excellent at fast changes but slowly "drifts" if used alone.

**DEFINITION**

**Gravity vector** — the direction gravity pulls (straight down toward Earth's centre). Its negative — the "up" direction — is what a still accelerometer actually reports.

Here is the key physical fact your steering exploits: a phone at rest feels only gravity, so the accelerometer's reading is the up direction, and the angle between "up" and the phone's own axes is the tilt. Conceptually:

teaching example

Python

```
1 # A still accelerometer reads the reaction to gravity, so its vector points UP.
2 up = -gravity           # gravity points down; "up" is its negative
3 # Tilt = the angle of the device's own axes relative to that up vector.
4 roll = atan2(up.x, up.y) # how far "up" leans left/right within the screen plane
```

This snippet embodies the single mechanical insight the whole steering system is built on:  $up = -gravity$ , and tilt is just the angle of the device relative to that up vector. Your real code never uses the raw accelerometer alone, though, because a phone in motion feels its own acceleration *plus* gravity, and the two are indistinguishable to one sensor. The fix is *sensor fusion*.

**DEFINITION**

**Sensor fusion** — combining several sensors (here gyroscope + accelerometer) so each covers the other's weakness — the gyro handles fast motion, the accelerometer anchors the long-term "down", and together they give a stable orientation.

Android ships this fusion as a virtual sensor. Your code asks for it by name, and crucially chooses the *magnetometer-free* variant:



```
apps/android-client/app/src/main/java/com/gamepad/client/MainActivity.kt · lines 263-265 Kotlin
1      rotationSensor =
2          sensorManager.getDefaultSensor(android.hardware.Sensor.TYPE_GAME_ROTATION_VECTOR)
3          ?: sensorManager.getDefaultSensor(android.hardware.Sensor.TYPE_ROTATION_VECTOR)
```

This is a deliberate physics decision. `TYPE_GAME_ROTATION_VECTOR` fuses only gyro and accelerometer; `TYPE_ROTATION_VECTOR` adds the magnetometer (the phone's compass). You *prefer* the first because a compass reads the Earth's magnetic field, which is warped by every speaker magnet, metal desk, and the PC beside you — it would inject heading errors into steering that has nothing to do with a compass heading. By dropping the magnetometer you trade away absolute north (which you never needed) for immunity to magnetic interference. The comment at lines 76-81 says exactly this: "no magnetometer → drift-free heading-independent, and crucially NO gravity blind-spot."

The fusion gives you an orientation as a rotation matrix; steering is then read straight off the gravity/up direction inside that matrix:

```
apps/android-client/app/src/main/java/com/gamepad/client/MainActivity.kt · lines 1257-1269 Kotlin
1      // World-up in the remapped view frame (3rd row). Level phone (screen vertical,
2      // facing you) → up ≈ (0, 1, 0).
3      val ux = fusionRemapped[6]
4      val uy = fusionRemapped[7]
5      val uz = fusionRemapped[8].coerceIn(-1f, 1f)
6      // ROLL = angle of up WITHIN the screen plane = left/right steering. Both legs
7      // (ux,uy) stay large in play holds → stable; ratio is preserved as you lean
8      // forward/back → decoupled from pitch; no gimbal-lock except screen-flat.
9      val rollDeg = STEER_SIGN * Math.toDegrees(Math.atan2(ux.toDouble(), uy.toDouble()))
10     // PITCH = how far up tips OUT of the screen plane = forward/back lean (3D look Y).
11     val pitchDeg = PITCH_SIGN * Math.toDegrees(Math.asin(uz.toDouble()))
12     fusedRollDeg = rollFilter.filter(rollDeg, event.timestamp).toFloat()
13     fusedPitchDeg = pitchFilter.filter(pitchDeg, event.timestamp).toFloat()
```

This is the mechanics of tilt made concrete. `fusionRemapped[6..8]` is the world "up" vector expressed in the phone's own frame. **Roll** (steering) is `atan2(up.x, up.y)` — how far up leans left or right *within* the screen plane. **Pitch** (forward/back lean, used for 3-D look) is `asin(up.z)` — how far up tips *out of* the screen plane. That is exactly the "angle of the device relative to gravity" from the concept snippet, now taken from a fused, jitter-resistant up vector instead of a noisy raw one.

#### DEFINITION

**Reference frame** — the set of axes you measure motion against. The same tilt looks different depending on whether you describe it in the phone's frame, the world's frame, or the display's frame.

Two subtleties here are pure physics-of-frames. First, `remapCoordinateSystem` (line 1255) rotates the world frame into a *landscape display* frame so that "zero steering" means the same physical pose regardless of which way round you hold the phone. Second — and this is the open risk in your shipped build — the constants `STEER_SIGN = 1.0` and `PITCH_SIGN = -1.0` (lines 94-95) choose the *sign convention* of each axis: whether tilting right produces a positive or negative roll.

**WATCH OUT**

`STEER_SIGN/PITCH_SIGN` are, per `SESSION_HANDOFF_2026-07-04.md` line 73, "device-unverified (guesses in shipped code)". A reference frame has no inherent "positive" direction; the physics fixes the *magnitude* of the angle but a human has to declare which way is positive. If tilt-right steers left on the device, the fix is to flip that one constant — a sign, not a math error.

**DEFINITION**

**Gimbal lock** — a failure of angle-based orientation where, at certain poses, two rotation axes line up and one degree of steering freedom collapses. The comment notes the fusion+remap keeps this singularity out of normal play holds.

**NOTE**

Sampling is set to `SENSOR_DELAY_FASTEST` (~200 Hz, line 1214) on a dedicated thread — deliberately faster than the display, so the steering readout never repeats a stale frame. That is a Nyquist-style choice: sample the motion faster than you display it, so you are never guessing between samples.

## Electromagnetism and radio: steering across an invisible wave

Once your phone knows its tilt, it must ship a 20-byte packet to the PC. Over Wi-Fi, that packet rides an *electromagnetic wave*, and the quality of that wave decides your latency.

**DEFINITION**

**Electromagnetic wave** — a self-propagating ripple of electric and magnetic field that travels at the speed of light. Radio, Wi-Fi, and visible light are all the same phenomenon at different frequencies.

**DEFINITION**

**Frequency band** — the range of wave frequencies a radio uses. Wi-Fi uses bands centred near 2.4 GHz, 5 GHz, and 6 GHz — billions of oscillations per second.

Your app surfaces the exact radio physics of the live link. It reads the connection's centre frequency in megahertz and classifies the band from it:

```
apps/android-client/app/src/main/java/com/gamepad/client/MainActivity.kt · lines 838-844 Kotlin

1          val band = when {
2              freq >= 5925 -> "6 GHz"
3              freq >= 4900 -> "5 GHz"
4              freq >= 2400 -> "2.4 GHz"
5              else -> "-"
6          }
7          "{\n\"band\": \"\$band\", \"freq\": \$freq, \"linkSpeed\": ${info.linkSpeed}, \"rssi\": ${info
.rssi}}"
```

Those numeric thresholds *are* electromagnetism: 2400, 4900, and 5925 MHz are the physical band edges of Wi-Fi radio. The same object reports `linkSpeed` (the negotiated data rate) and `rssi` — the received signal

strength — which the UI renders as decibel-milliwatts:

```
apps/controller-ui/src/app/App.tsx · lines 3278-3279 TypeScript · React
1      <p className="text-[10px] uppercase tracking-wide text-muted-foreground">Signal</p>
2      <p className="text-sm font-bold text-foreground">{wifi.rssi < 0 ? `${wifi.rssi} dBm` : "-"}</p>
```

#### DEFINITION

**RSSI / dBm** — RSSI is received signal strength; dBm is a logarithmic power scale where more-negative means weaker. A reading of -64 dBm is a strong, close link; -85 dBm is weak and far.

This card embodies why radio physics matters for a controller. Your **LatencyCard** comment (line 3242) states it flatly: "2.4 vs 5 GHz is the #1 cause of latency gaps between phones." The reason is physical. The 2.4 GHz band is narrow, crowded, and shared with microwaves, Bluetooth, and neighbours' routers — the **LOW\_LATENCY\_GUIDE.md** names exactly those interferers. That congestion doesn't slow the wave (light-speed is fixed) but it forces retransmissions and queuing, which show up as *jitter* — the variance in latency that ruins a real-time input feel. The 5 GHz band is wider and less crowded, so packets clear faster and more predictably. Your guide's latency table makes the tradeoff quantitative: tuned 5 GHz reaches ~10-18 ms while congested 2.4 GHz balloons to 50-120 ms+. That is why the UI paints 2.4 GHz amber and nudges the user to switch — a physics warning dressed as a UI hint.

#### EXPERT DETAIL

A weaker RSSI forces the radio to fall back to a slower, more robust modulation to keep bit-errors down, which lowers **linkSpeed** — so signal strength and link rate are physically coupled, and both feed the jitter you feel. This is why the card shows all three (band, link, signal) together rather than any one alone.

## Electrical signalling over the wire

When you switch to USB, the wave is gone and the physics becomes *conducted electrical signalling* down a cable. USB moves bits as a *differential signal*: two wires carrying mirror-image voltages, where the receiver reads the *difference* between them so that shared electrical noise cancels out. That is why a USB link is quieter and lower-latency than radio — the **LOW\_LATENCY\_GUIDE.md** puts it at ~6-12 ms, the lowest tier in the table.

#### DEFINITION

**Differential signalling** — sending one bit as the voltage *difference* between a pair of wires. Noise hitting both wires equally cancels in the subtraction, giving a clean, fast signal.

Your **aoa\_transport.py** speaks this layer directly. It negotiates the Android Open Accessory handshake with USB *control transfers* (vendor requests on the device), then claims raw *bulk endpoints* to stream packets:

apps/pc-server/aoa\_transport.py · lines 232-239

Python

```
1 ep_in = usb.util.find_descriptor(  
2     intf, custom_match=lambda e:  
3     usb.util.endpoint_direction(e.bEndpointAddress) == usb.util.ENDPOINT_IN  
4     and usb.util.endpoint_type(e.bmAttributes) == usb.util.ENDPOINT_TYPE_BULK)  
5 ep_out = usb.util.find_descriptor(  
6     intf, custom_match=lambda e:  
7     usb.util.endpoint_direction(e.bEndpointAddress) == usb.util.ENDPOINT_OUT  
8     and usb.util.endpoint_type(e.bmAttributes) == usb.util.ENDPOINT_TYPE_BULK)
```

An *endpoint* is a physical data channel on the USB device; **ENDPOINT\_IN** carries phone→PC input packets and **ENDPOINT\_OUT** carries the PC→phone ACK/rumble. The file's header targets "~1-2 ms" wired latency by talking to these endpoints with "no adb, no tethering, no IP stack" — stripping away every software layer between the byte and the wire. This is the electrical-signalling branch at its purest: the same 20-byte payload, but riding conductor voltage instead of a radio wave.

#### NOTE

The shipped USB path today is RNDIS *tether*, not raw AOA. `MainActivity.kt` (`usbBroadcastAddress`, lines 143-160) hunts for an `rndis/usb` interface and routes UDP over it. RNDIS still rides the USB cable's differential wires — you get the electrical-noise immunity — but the packet is wrapped in a full IP stack, so it sits between Wi-Fi and raw AOA in latency. AOA (the ~1-2 ms path above) is coded but parked at a driver gate, per the handoff.

## Thermodynamics: heat, throttling, and the seal-gate win

Every calculation your phone does dissipates energy as heat, and heat is the enemy of consistency. When a chip gets too hot it *thermal-throttles* — it deliberately slows itself to avoid damage — and a throttled CPU wakes your 1000 Hz send loop late, which you feel as latency jitter. This is why thermodynamics is a first-class concern in a real-time input app.

#### DEFINITION

**Thermal throttling** — a chip lowering its own clock speed when it overheats, trading performance for safety. On a phone it causes scheduler jitter and downclocking — exactly what `LOW_LATENCY_GUIDE.md` warns about.

Your UI reads the phone's temperature (from the battery sensor, `getSystemStatsJson`, ~38 C in normal play) and turns it into a three-tier thermal state:

apps/controller-ui/src/app/App.tsx · lines 2790-2794

TypeScript · React

```
1 const status = temp < 38  
2   ? { label: "NOMINAL", c: "text-emerald-400", b: "bg-emerald-400", desc: "Cooling efficient. Power and r  
   efresh speeds are fully maximized." }  
3   : temp < 44  
4     ? { label: "OPTIMIZED", c: "text-amber-400", b: "bg-amber-400", desc: "Balanced thermal levels. Syste  
       m throttling auto-managed." }
```

apps/controller-ui/src/app/App.tsx · (cont.)

TypeScript · React

```

5      : { label: "ELEVATED", c: "text-red-400", b: "bg-red-400", desc: "Bypass mode active to cool down pro
      cessor immediately." };

```

The 38 C and 44 C thresholds are a thermodynamic model of the device: below 38 the chip runs unthrottled; the "OPTIMIZED / Balanced thermal levels" band means the system is auto-managing throttling; above 44 it is actively shedding heat. (The default temperature stub in `MainActivity.kt` is 38.4f, line 493 — the code, not any round-number doc, is the source of truth.)

Two of your biggest performance wins were fought on thermodynamic grounds, not algorithmic ones. First, CPU-affinity pinning was *removed*:

apps/android-client/app/src/main/cpp/gamepad-engine.cpp · lines 212-216

C++

```

1      // NOTE: no CPU-affinity pinning. Forcing cores 4-7 (big cores) kept them spun
2      // up 1000x/second, which was the main source of high CPU + phone heat. This
3      // thread is now light (it mostly sleeps), so the scheduler can keep it on an
4      // efficiency core. (Affinity pinning was intentionally removed for thermal
5      // reasons — do NOT re-add it.)

```

This is a direct heat/energy trade: pinning the send loop to the powerful "big" cores kept them energised a thousand times a second, generating heat that eventually throttled everything. Letting the loop drift to an efficiency core cut the energy draw. The second, larger win gates the GRX encryption seal so it only runs on frames you actually transmit:

apps/android-client/app/src/main/cpp/gamepad-engine.cpp · lines 304-309

C++

```

1      // Only seal when we're actually transmitting this tick. The seal is a JNI
2      // up-call into Kotlin AES-GCM plus heap allocations; gating it on grxReady
3      // ALONE (the old code) ran it on every idle 2ms poll — ~500 seals/sec whose
4      // output was discarded because the send below is gated on doSend. Gating on
5      // doSend cuts that to only the frames we send (change + redundancy + keep-alive).
6      if (doSend && grxReady.load() && g_mid_seal && g_activity) {

```

`SESSION_HANDOFF_2026-07-04.md` line 62 calls this "the biggest heat/jitter win." The physics is straightforward: an AES-GCM seal is real computational work, and work is heat. The old code did ~500 seals per second and *threw the results away* because the actual send is gated on `doSend`. Gating the seal on the same condition eliminated hundreds of pointless encryptions per second — less wasted energy, less heat, less throttling, and therefore less jitter. It is a thermodynamics optimisation wearing a cryptography costume, and it ties this section back to the first: the whole point of running cool is to keep that 200 Hz tilt reading and 1000 Hz packet loop firing on time.

---

DISCIPLINE 3

# Electrical & Computer Engineering

3

## The Discipline, and Where It Hides

Electrical & Computer Engineering (ECE) is the field that sits between physics and software — it studies how to build and program the machinery that actually runs code: logic gates, processors, memory, and the timing that governs them. GamepadOS is a software project, but its performance-critical core is written the way an embedded engineer would write it, and that shows.

### DEFINITION

**Digital logic** — the design of circuits that compute with two-valued signals (0 and 1). Every AND, OR, and shift your code performs is ultimately a pattern of logic gates switching.

### DEFINITION

**Computer architecture** — the study of how a processor is organized — its instruction set, registers, memory, and cache — and how software must be shaped to run well on it.

### DEFINITION

**Real-time systems** — systems where being *correct* also means being *on time*. A result that is right but late is a failure. A game controller is a soft real-time system.

## Computer Architecture — One Program, Three Instruction Sets

A compiled program is machine code for one specific **instruction-set architecture (ISA)** — the "dialect" a particular CPU family speaks. Phones and emulators use different ISAs, so the native engine is compiled three times and bundled together:

apps/android-client/app/build.gradle.kts · lines 26-41

Kotlin

```

1      ndk {
2          // Broad device coverage (universal APK for sideload + in-app updater):
3          // arm64-v8a — all modern phones/tablets (primary)
4          // armeabi-v7a — older / budget 32-bit ARM phones
5          // x86_64 — Android emulators, Chromebooks, x86 tablets
6          // (32-bit x86 omitted: effectively dead, only adds size.)
7          abiFilters.add("arm64-v8a")
8          abiFilters.add("armeabi-v7a")
9          abiFilters.add("x86_64")
10     }
11
12     externalNativeBuild {
13         cmake {
14             cppFlags += listOf("-O3", "-flto", "-ffast-math")

```

apps/android-client/app/build.gradle.kts · (cont.)

Kotlin

```

15         }
16     }

```

**DEFINITION**

**ABI / ISA** — an Application Binary Interface pins down how compiled code is laid out and called for a given CPU. `arm64-v8a` is 64-bit ARM (modern phones); `armeabi-v7a` is 32-bit ARM (old phones); `x86_64` is Intel/AMD (emulators). Each needs its own machine code.

The `cppFlags` are architecture-aware compiler optimizations, and they're a direct ECE concern: `-O3` is maximum optimization, `-flto` is link-time optimization (letting the compiler optimize across file boundaries), and `-ffast-math` relaxes floating-point strictness for speed. These flags tell the compiler to squeeze the tightest, fastest machine code out of the source — exactly the trade an engineer makes when the hot path must be fast.

## Memory Layout — Bytes Where You Put Them

Computer architecture is also about *how data sits in memory*, and the wire packet is defined with byte-exact control that a systems engineer would recognize immediately:

apps/android-client/app/src/main/cpp/gamepad-engine.cpp · lines 51-65

C++

```

1  #pragma pack(push, 1)
2  struct GamepadPayload {
3      uint64_t timestamp; // 8 bytes
4      uint16_t buttons; // 2 bytes
5      uint8_t leftTrigger; // 1 byte
6      uint8_t rightTrigger; // 1 byte
7      uint8_t leftStickX; // 1 byte
8      uint8_t leftStickY; // 1 byte
9      uint8_t rightStickX; // 1 byte
10     uint8_t rightStickY; // 1 byte
11     uint32_t authToken; // 4 bytes
12 };
13 #pragma pack(pop)
14
15 static_assert(sizeof(GamepadPayload) == 20, "FATAL: Payload struct is not exactly 20 bytes!");

```

**DEFINITION**

**Struct padding / alignment** — CPUs prefer data aligned to certain byte boundaries, so compilers silently insert *padding* bytes between fields. That padding would change the packet's size and break the phone↔PC contract.

`#pragma pack(1)` orders the compiler to use **no padding** — pack the fields tight, byte after byte. The `static_assert(sizeof(...) == 20)` is a **compile-time** guarantee: if any change ever makes the struct not exactly 20 bytes, the build fails before a single wrong packet can be sent. This is architecture-aware programming — knowing that the machine wants to pad, and forbidding it — plus the fixed-width integer types (`uint64_t`, `uint16_t`, `uint8_t`) that specify exact sizes rather than letting the platform



decide. The engine also **pre-allocates** its state globally (the atomics at the top of the file) so the hot path never pauses to ask the OS for memory.

## Real-Time Systems — Being Right and On Time

The defining ECE idea in GamepadOS is **real-time scheduling**. The send loop must wake up on schedule, every time, or input arrives late and jittery. So the engine asks the operating system for real-time priority:

```
apps/android-client/app/src/main/cpp/gamepad-engine.cpp · lines 223-244 C++

1  {
2      struct sched_param sp;
3      memset(&sp, 0, sizeof(sp));
4      // Use a low RT priority: just above normal, well clear of audio/driver
5      // threads. sched_get_priority_min gives the floor for SCHED_FIFO.
6      int rtMin = sched_get_priority_min(SCHED_FIFO);
7      sp.sched_priority = (rtMin > 0) ? rtMin + 1 : 1;
8      int rc = pthread_setschedparam(pthread_self(), SCHED_FIFO, &sp);
9      if (rc != 0) {
10         // SCHED_FIFO denied (typical on Android without CAP_SYS_NICE).
11         // Fall back to the strongest CFS niceness we can get. setpriority
12         // returns -1 on failure but -1 is also a valid niceness, so clear
13         // errno first and check it afterwards.
14         errno = 0;
15         if (setpriority(PRIO_PROCESS, 0, -19) != 0 && errno != 0) {
16             LOGI("TX thread priority bump unavailable (SCHED_FIFO rc=%d, setpriority errno=%d); running
17 at default.", rc, errno);
18         } else {
19             LOGI("TX thread niced to high CFS priority (SCHED_FIFO denied rc=%d).", rc);
20         }
21     } else {
22         LOGI("TX thread elevated to SCHED_FIFO prio %d.", sp.sched_priority);
23     }
24 }
```

### DEFINITION

**SCHED\_FIFO** — a real-time scheduling policy in the Linux/Android kernel. A SCHED\_FIFO thread runs ahead of ordinary ("CFS") threads and isn't casually preempted — the OS promises to schedule it promptly, which is what "real-time" buys you.

### DEFINITION

**Jitter** — variation in timing. A loop that runs every 1 ms on average but sometimes takes 5 ms has jitter, and jitter is what makes input feel inconsistent even when average latency is low.

This is textbook real-time engineering. It *requests* SCHED\_FIFO (the strongest timing guarantee), but knows Android usually denies it without special capability (**CAP\_SYS\_NICE**), so it **degrades gracefully** to the best ordinary priority it can get (**nice -19**), and if even that fails, it just runs normally and logs it. The whole block "must degrade gracefully — never crash, never block." That defensive, layered pursuit of predictable timing — chase the ideal, fall back cleanly — is exactly how embedded and real-time engineers think.

## Signalling for Priority — TOS and WMM

ECE also reaches into how the packet is *tagged* for the network hardware. The engine marks its datagrams as high-priority at two levels:

```
apps/android-client/app/src/main/cpp/gamepad-engine.cpp · lines 135-148 C++

1 static void applyLowLatencyTos(int sock) {
2     if (sock < 0) return;
3     int tos = 0xB8; // DSCP EF (Expedited Forwarding) << 2 | ECN 0
4     setsockopt(sock, IPPROTO_IP, IP_TOS, &tos, sizeof(tos));
5     // Android/Linux Wi-Fi WMM access-category selection is driven by skb->priority
6     // (SO_PRIORITY), which many Wi-Fi HALs honour even when they ignore IP_TOS/DSCP.
7     // prio 6 (TC_PRIO_INTERACTIVE) maps to WMM AC_VO – the same high-priority queue the
8     // WIFI_MODE_FULL_LOW_LATENCY lock targets, so they compound and cut uplink queuing
9     // jitter on busy APs. Best-effort: prio <= 6 is allowed unprivileged; failure is
10    // harmless (older kernels may not define SO_PRIORITY).
11    #ifdef SO_PRIORITY
12        int prio = 6;
13        setsockopt(sock, SOL_SOCKET, SO_PRIORITY, &prio, sizeof(prio));
14    #endif
```

`IP_TOS = 0xB8` sets the DSCP "Expedited Forwarding" class in the IP header, and `SO_PRIORITY = 6` sets the socket priority that maps to the Wi-Fi radio's **AC\_VO** (voice) access category — the fastest of Wi-Fi's four hardware queues. This is a hardware-aware move: the engineer knows the Wi-Fi chip sorts outgoing packets into priority queues, and deliberately places controller input in the express lane. Understanding that the network interface is a piece of prioritizing hardware, not a transparent pipe, is an ECE insight.

## Where Each Branch Lives

| ECE sub-field               | Where in GamepadOS                                 |
|-----------------------------|----------------------------------------------------|
| Digital logic               | the bitmask, exact-width integer types             |
| Computer architecture (ISA) | the three ABIs, <code>-O3/-flto</code> build flags |
| Memory layout / alignment   | <code>#pragma pack(1) + static_assert(==20)</code> |
| Real-time systems           | the SCHED_FIFO TX thread + graceful fallback       |
| Hardware-aware networking   | IP_TOS / SO_PRIORITY WMM tagging                   |

The controller UI and website are ordinary application software, but the native engine is embedded systems engineering: exact memory layout, real-time scheduling, and hardware-aware tagging, all in pursuit of one thing an application programmer rarely worries about — being reliably on time.

---

DISCIPLINE 4

# Signal Processing & Control Theory

4

## The Discipline, and Where It Hides

Signal processing and control theory are the two engineering sciences of *streams over time*. Signal processing asks "how do I clean up and interpret a stream of measurements?"; control theory asks "how do I shape an input to steer a system the way I want?" GamepadOS is a machine for turning noisy human motion into a steady stream of controller values, so both fields are woven through its input path.

### DEFINITION

**Signal processing** — the analysis and modification of signals — streams of values sampled over time — to reduce noise, extract meaning, or change their form. Filtering a jittery gyroscope is signal processing.

### DEFINITION

**Control theory** — the mathematics of systems that respond to inputs, especially shaping an input (through curves, deadzones, and feedback) so the output behaves well. Tuning how a tilt becomes a turn is control theory.

### DEFINITION

**Sampling** — measuring a continuous quantity at regular instants. A 1000 Hz sample rate means one measurement every millisecond.

## Sampling Theory — Turning Motion Into a Stream

The whole system is a **sampler**: it reads your continuous, physical motion at a fixed rate and emits discrete packets. The engine samples and sends at up to 1000 Hz — one packet per millisecond — which the app advertises as "Engine locked at 1000 Hz · ultra-low latency."

### DEFINITION

**Sample rate & period** — the rate is how many samples per second (Hz); the period is the time between them (period =  $1 / \text{rate}$ ). 1000 Hz  $\square$  a 1 ms period.

Why so fast? Sampling theory says that to represent motion faithfully you must sample well above the fastest change you care about; sample too slowly and quick flicks are missed or blur (a phenomenon related to *aliasing*). A human thumb and wrist can't move at 1000 Hz, so 1000 samples per second captures every nuance with margin to spare — the discrete stream feels perfectly continuous. The engine is also **event-driven**: it wakes to send only when the input actually changed (plus a low-rate keep-alive), and the keep-alive itself adapts — rising from 30 to 60 Hz while rumble is active. Adapting the sample/emit rate to what's happening is a signal-processing sensibility: spend bandwidth where information is, save it where there's none.

## Digital Filtering — the One-Euro Low-Pass

Raw gyroscope data is noisy; if you fed it straight to the game the view would shake. The classic fix is a **low-pass filter** (let slow movements through, damp fast jitter), but a fixed low-pass forces a bad trade: smooth enough to kill jitter *and* it lags behind fast motion. GamepadOS uses the **One-Euro filter**, an adaptive low-pass whose smoothing changes with how fast you're moving:

```
apps/android-client/app/src/main/java/com/gamepad/client/MainActivity.kt · lines 1300-1318 Kotlin

1      private fun alpha(cutoff: Double, dt: Double): Double {
2          val tau = 1.0 / (2.0 * Math.PI * cutoff)
3          return 1.0 / (1.0 + tau / dt)
4      }
5      fun filter(x: Double, tNs: Long): Double {
6          if (!started) { started = true; xPrev = x; dxPrev = 0.0; tPrevNs = tNs; return x }
7          var dt = (tNs - tPrevNs) / 1_000_000_000.0
8          if (dt <= 0.0 || dt > 0.1) dt = 1.0 / 200.0 // guard bad/large gaps
9          tPrevNs = tNs
10         val dx = (x - xPrev) / dt
11         val aD = alpha(dCutoff, dt)
12         val edx = aD * dx + (1 - aD) * dxPrev
13         dxPrev = edx
14         val cutoff = minCutoff + beta * Math.abs(edx)
15         val aX = alpha(cutoff, dt)
16         val xFilt = aX * x + (1 - aX) * xPrev
17         xPrev = xFilt
18         return xFilt
19     }
```

This is a textbook exponential low-pass with a twist. The `alpha` function computes the classic smoothing factor from a **cutoff frequency** and the real time step `dt`:  $\tau = 1/(2\pi \cdot \text{cutoff})$  is the filter's time constant, and  $\alpha = 1/(1 + \tau/\text{dt})$ . The line `val xFilt = aX * x + (1 - aX) * xPrev` is **exponential smoothing** — a weighted blend of the new reading and the previous output. The clever part is `val cutoff = minCutoff + beta * Math.abs(edx)`: the cutoff *rises with the estimated speed* (`edx`), so when your hand is still the cutoff is low (heavy smoothing, no jitter) and when you flick fast the cutoff jumps (light smoothing, no lag). That's the One-Euro filter (Casiez, Roussel & Vogel, 2012), named in the source, tuned to 2.8 Hz /  $\beta = 0.5$ .

### DEFINITION

**Cutoff frequency** — the frequency above which a low-pass filter starts strongly attenuating. Lower cutoff = smoother but laggier; higher cutoff = more responsive but noisier. The One-Euro filter *moves* the cutoff based on speed to get the best of both.

### EXPERT DETAIL

The derivative is itself low-passed (`aD * dx + (1 - aD) * dxPrev`) before it drives the cutoff, so a single noisy sample can't spike the responsiveness. And `dt` is measured per real sample with a guard against bad gaps — a **frame-rate-independent** formulation, so the filter behaves the same whether the sensor delivers at 100 or 200 Hz. The browser preview mirrors this with `alpha = 1 - exp(-rate*dt)`, the continuous-time equivalent, so the on-screen feel matches the device.

## Control Shaping — Deadzones and Response Curves

Once the signal is clean, control theory shapes it into a *feel*. Two shaping functions dominate. The first is the **deadzone**, which ignores tiny inputs so the character doesn't drift, done as a smooth "subtract-and-rescale" rather than a hard gate:

```
apps/controller-ui/src/app/App.tsx · lines 924-935 TypeScript · React

1  const processStick = (x: number, y: number) => {
2    const distance = Math.sqrt(x * x + y * y);
3    const deadzoneRadius = (8 / 100) * 60; // 8% default deadzone
4    if (distance >= deadzoneRadius && distance > 0) {
5      const factor = (distance - deadzoneRadius) / (60 - deadzoneRadius);
6      return {
7        x: (x / distance) * factor * 60,
8        y: (y / distance) * factor * 60
9      };
10   }
11   return { x: 0, y: 0 };
12 }
```

This is an **input-shaping transfer function**: below the deadzone radius the output is zero; above it, the magnitude is *rescaled* so it ramps from 0 at the edge up to full — no sudden jump. In control terms it's a piecewise transfer curve designed to reject small disturbances (hand tremor, sensor noise) while preserving full range. The second shaping function is **expo**, a response curve that blends linear and cubic terms so control is fine near center and fast at the extremes — a deliberately nonlinear gain, exactly the kind of curve control engineers design to match a system's response to human intent.

## Feedback Loops — Closing the Circuit

Control theory's signature idea is the **feedback loop**, and GamepadOS closes one through the human. Your input moves the virtual pad; the game reacts; and the game's rumble is sent *back* to the phone, which vibrates — so you feel the consequence of your action and adjust. The encrypted transport even reserves a return direction for it (`DIR_S2C = 2`, "server → client rumble/ack"). The full circuit is: thumb → packet → game → rumble → thumb. That is a closed-loop control system with a human operator in the loop, and it's why haptics aren't a gimmick — they close the feedback path that a physical controller closes through your fingers naturally.

## Where Each Branch Lives

| Sub-field                         | Where in GamepadOS                                     |
|-----------------------------------|--------------------------------------------------------|
| Sampling theory                   | 1000 Hz engine, adaptive keep-alive, event-driven send |
| Digital filtering (low-pass)      | the One-Euro gyro filter (2.8/0.5)                     |
| Control shaping (transfer curves) | the smooth deadzone + expo response curve              |

| Feedback / closed-loop                                                                                                                                                                                                                                                                     | input → game → rumble → hand |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------|
| <p>The input path is, from end to end, a small signal-processing pipeline followed by a control system: sample fast, filter adaptively, shape the response, and close the loop with feedback. Every one of those stages is a deliberate engineering choice from these two disciplines.</p> |                              |

---

DISCIPLINE 5

# Networking & Distributed Systems

5



## The Discipline, and Where It Hides

Computer networking is the study of how independent machines exchange data reliably (or deliberately unreliably) across a shared medium; distributed systems is the broader study of how separate computers cooperate to act like one. GamepadOS is fundamentally two machines — a phone and a PC — that must agree, discover each other, and stay in sync, so these two fields are its backbone.

### DEFINITION

**Network stack** — the layered set of rules that carry data from an app on one machine to an app on another — addresses to find the machine, ports to find the program, and a transport to move the bytes.

### DEFINITION

**Protocol** — an agreed format and sequence of messages two machines use to understand each other. The 20-byte packet plus its handshake is GamepadOS's protocol.

### DEFINITION

**Distributed system** — multiple computers coordinating over a network toward a shared goal, facing problems a single machine never has — partial failure, message loss, and keeping copies consistent.

## Addresses, Ports, and Subnets — Finding the Other Machine

Before any data flows, the phone must locate the PC. That uses the two fundamental identifiers of the network stack, both just numbers:

### DEFINITION

**IP address** — a 32-bit number (written like **192.168.1.7**) that identifies a machine on the network. A subnet like **/24** means the first 24 bits are the shared network prefix and the last 8 identify the host — so it's binary masking of the address.

### DEFINITION

**Port** — a 16-bit number (0–65535) that identifies which program on a machine should receive the data. GamepadOS's server listens on UDP port **7777**.

The server picks which of its IP addresses to advertise in the pairing QR, deliberately preferring a real LAN address (192.168.x) over a virtual-adaptor one (10.x) so pairing keeps working even behind a VPN — an addressing subtlety that only makes sense once you know an IP is a routable number on a specific subnet. The phone then builds the destination address from the scanned IP and port:

apps/android-client/app/src/main/cpp/gamepad-engine.cpp · lines 698-701

C++

```

1  memset(&serverAddr, 0, sizeof(serverAddr));
2  serverAddr.sin_family = AF_INET;
3  serverAddr.sin_port = htons(port);
4  inet_pton(AF_INET, ip, &serverAddr.sin_addr);

```

`sin_family = AF_INET` selects IPv4, `htons(port)` writes the 16-bit port in network byte order (**host-to-network short** — endianness again, because the network standard is big-endian), and `inet_pton` parses the dotted-decimal IP string into its 32-bit form. This is the network stack's addressing layer, built by hand.

## UDP vs TCP — Choosing to Be Unreliable

The single most consequential networking decision in the project is the **transport**: GamepadOS sends input over **UDP**, not TCP, and that choice is pure distributed-systems reasoning.

### DEFINITION

**TCP** — a reliable, ordered transport — it guarantees every byte arrives, in order, resending lost data. That reliability costs time: a lost packet stalls everything behind it ("head-of-line blocking").

### DEFINITION

**UDP** — an unreliable, connectionless transport — it fires a datagram and forgets it. No resending, no ordering, no stalls. If a packet is lost, it's simply gone.

For a controller, UDP is *correct* precisely because it's unreliable. A dropped input frame is worthless a millisecond later — the next frame already supersedes it — so re-sending a stale input (as TCP would) is worse than losing it. The engine creates a UDP socket, makes it non-blocking, and enables broadcast for discovery:

apps/android-client/app/src/main/cpp/gamepad-engine.cpp · lines 686-693

C++

```

1  udpSocket = socket(AF_INET, SOCK_DGRAM, 0);
2  if (udpSocket != -1) {
3      int flags = fcntl(udpSocket, F_GETFL, 0);
4      fcntl(udpSocket, F_SETFL, flags | O_NONBLOCK);
5      int broadcastEnable = 1;
6      setsockopt(udpSocket, SOL_SOCKET, SO_BROADCAST, &broadcastEnable, sizeof(broadcastEnable));
7      applyLowLatencyTos(udpSocket);
8  }

```

`SOCK_DGRAM` is UDP (a datagram socket); `O_NONBLOCK` means a send never waits — if the buffer is full the call returns immediately rather than stalling the real-time loop; and `SO_BROADCAST` lets the phone shout to the whole subnet during discovery. On the server side the mirror of this is one line: `sock.bind(('0.0.0.0', port))` — bind to all interfaces on the chosen port and listen. Two sockets, one datagram protocol, no reliability layer: a deliberate design that trades guaranteed delivery for guaranteed freshness.

## Discovery, Lock-On, and Failure Detection

The phone and PC form a **client/server pair**, and their connection lifecycle is classic distributed-systems engineering: discover, lock on, keep alive, detect failure. The phone starts by *broadcasting* (shouting to the subnet via `sendto`) until the server replies with an ACK; then it **locks on** to that peer and switches from broadcast `sendto()` to a `connect()`ed `send()` — unicast to one address, faster and cleaner. Liveness is then maintained the way every distributed system maintains it: with a heartbeat and a watchdog.

#### DEFINITION

**Heartbeat & watchdog** — a heartbeat is a periodic "I'm still here" message; a watchdog is a timer that declares the peer dead if no heartbeat arrives in time. GamepadOS uses a low-rate keep-alive plus the server's ~3-second idle watchdog to drop stale sessions.

#### DEFINITION

**Failure detection** — deciding, without a reliable signal, that a remote machine has gone away. Over a lossy UDP link you can never be *sure* a peer is gone — you infer it from silence, which is why the watchdog (not an explicit goodbye) is the source of truth.

Notably, the project *removed* an explicit "BYE" teardown packet and now relies solely on the watchdog — an important distributed-systems lesson: over an unauthenticated network, an explicit teardown message is both unreliable (it can be lost) and a security hole (it can be forged), whereas inferring death from silence is robust.

## Versioning as a Consistency Problem

The subtlest distributed-systems idea in GamepadOS isn't in the packet path at all — it's in the updater. The phone, the PC server, and the website each hold a version number, and they must agree on which is newest. That is a **consistency** problem: multiple copies of a fact that can drift apart.

#### DEFINITION

**Consistency (distributed systems)** — the property that all parts of a system agree on shared state. When the same fact (the current version) lives in several places, keeping them consistent is a real engineering task — and letting them diverge causes the update loops documented in the project's history.

The solution is a **single source of truth**: the website's `/api/version` endpoint reads the one active release from the database and computes the download's checksum live, so the version advertised, the file served, and its hash can never disagree. The `versionCode` is required to increase monotonically so "newer" is unambiguous. These are the same techniques distributed databases use to avoid split-brain — one authority, monotonic ordering — applied to a tiny three-node system.

## Where Each Branch Lives

Sub-field

Where in GamepadOS

|                                  |                                                                          |
|----------------------------------|--------------------------------------------------------------------------|
| Addressing (IP, ports, subnets)  | QR IP selection, <code>htons/inet_pton</code> , port 7777                |
| Transport choice (UDP vs TCP)    | <code>SOCK_DGRAM</code> non-blocking socket, drop-don't-resend           |
| Discovery & connection lifecycle | broadcast → ACK → lock-on → unicast                                      |
| Failure detection                | keep-alive + idle watchdog, no BYE packet                                |
| Distributed consistency          | single-source-of-truth versioning, monotonic<br><code>versionCode</code> |

Two machines that have never met, on a lossy shared medium, agreeing to act as one low-latency controller — that is a distributed system in miniature, and nearly every classic networking decision (unreliable transport, broadcast discovery, heartbeat liveness, one authoritative copy) shows up in it.

---

DISCIPLINE 6

# Cryptography & Security

6

There is a moment in GamepadOS that most projects never earn: your phone and your PC, having never spoken before, agree on a secret that an attacker sitting on the same Wi-Fi cannot compute, and from then on every 20-byte button-press is sealed so that the same attacker can neither read it nor forge it nor replay it. That single capability is not one idea. It is a stack of distinct academic disciplines, each with centuries of theory behind it, all of which you assembled inside `apps/pc-server/grx_crypto.py` and its Kotlin and Swift twins. This section names each discipline, then points at the exact line of your own code where it lives.

**DEFINITION**

**Cryptography** — the science of designing communication that stays confidential and unforgeable even when your adversary sees every byte on the wire and knows exactly how your system works. Its security must rest on secret keys, never on secret methods.

**DEFINITION**

**Security engineering** — the broader discipline of building systems that hold up against a deliberate adversary — choosing a threat model, finding the weakest link, and closing it — of which cryptography is one tool among many.

The interesting thing about your GRX layer is that it is a *thin* piece of code (a few hundred lines) sitting on top of a *deep* mountain of mathematics. Let us climb the mountain first, then see how little of it you had to re-derive.

| Sub-branch of study              | Where it lives in GamepadOS                                                           |
|----------------------------------|---------------------------------------------------------------------------------------|
| Number theory / finite fields    | X25519 arithmetic mod the prime $2^{255}-19$                                          |
| Group theory (elliptic curves)   | the Diffie-Hellman key exchange, <code>my_priv.exchange</code>                        |
| Abstract algebra (Galois fields) | the GCM authentication tag, $GF(2^{128})$                                             |
| Probability                      | SHA-256 collision resistance, the birthday bound                                      |
| Applied cryptography (AEAD)      | <code>AESGCM.encrypt</code> , the nonce, the never-reset counter                      |
| Key derivation                   | HKDF-SHA256 in <code>derive_keys</code> / <code>deriveKeys</code> / <code>hkdf</code> |
| Threat modeling / replay         | the sliding <code>ReplayWindow</code> , verify-tag-first                              |

## The mathematics underneath: number theory and the prime field

**DEFINITION**

**Number theory** — the study of the integers — divisibility, primes, and arithmetic done "on a clock" that wraps around after some modulus. It is the oldest branch of pure mathematics and the foundation of nearly all modern public-key cryptography.

**DEFINITION**

**Finite field (prime field)** — the set of integers  $\{0, 1, \dots, p-1\}$  for a prime  $p$ , where you can add, subtract, multiply, and — crucially — *divide* (every nonzero element has a reciprocal), all modulo  $p$ . It behaves like ordinary arithmetic but lives inside a finite box.

Every X25519 key in your project is a point whose coordinates are elements of one specific finite field: the integers modulo the prime  $p = 2^{255} - 19$ . That prime is not arbitrary — it is chosen to be just under a power of two so the modular reductions are fast, and the whole curve is named *Curve25519* after it. When your PC does `my_priv.exchange(their_pub)`, the machine is multiplying a 255-bit secret scalar into a point and reducing every intermediate coordinate modulo  $2^{255}-19$ .

**DEFINITION**

**Group / elliptic-curve group** — a set with one operation (here, "adding" two points on a curve) that is associative, has an identity element, and gives every element an inverse. On Curve25519 the points form such a group, and "multiplying a point by a scalar  $k$ " just means adding the point to itself  $k$  times.

**DEFINITION**

**Diffie-Hellman key exchange** — a method where each side picks a private scalar, publishes the corresponding public point, and both multiply their own scalar into the other's point. Because scalar multiplication commutes ( $a \cdot (b \cdot G) = b \cdot (a \cdot G)$ ), the two sides land on the *same* secret point — without ever transmitting a secret.

apps/pc-server/grx\_crypto.py · lines 73-82

Python

```

1 def handshake(my_priv, their_pub_bytes, psk, long_term_id, *, am_client):
2     """Run the full key agreement from one side. Returns a dict of keys + transcript."""
3     my_pub = my_priv.public_key().public_bytes_raw()
4     their_pub = X25519PublicKey.from_public_bytes(their_pub_bytes)
5     shared = my_priv.exchange(their_pub)
6     cli, srv = (my_pub, their_pub_bytes) if am_client else (their_pub_bytes, my_pub)
7     th = transcript_hash(cli, srv, long_term_id)
8     keys = derive_keys(shared, psk, th)
9     keys['transcript'] = th
10    return keys

```

The single line `shared = my_priv.exchange(their_pub)` is the entire elliptic-curve group operation. It embodies number theory (arithmetic mod  $2^{255}-19$ ) *and* group theory (scalar multiplication on the curve). The same line appears in Kotlin as `X25519.computeSharedSecret(ephPriv, serverPub)` and in Swift as `ephPriv.sharedSecretFromKeyAgreement(with: pub)` — three languages, one piece of 1970s mathematics.

**DEFINITION**

**Discrete logarithm problem** — given the public point  $Q = k \cdot G$  and the base point  $G$ , recover the secret scalar  $k$ . On a well-chosen elliptic curve there is no known method faster than roughly  $2^{128}$  operations — computationally impossible. This *hardness assumption* is the load-bearing wall under the entire handshake.

Your security rests on a belief that nobody can *reverse* that multiplication. The attacker sees both public points fly by in the clear (`GRX_PROTOCOL.md` step 1: "exchange ephemeral public keys (32 B each), in the clear"), yet cannot recover either private scalar, so cannot compute *shared*. That is the discrete-log assumption doing all the work.

## Abstract algebra in the tag: the Galois field $GF(2^{128})$

**DEFINITION**

**Abstract algebra** — the study of algebraic *structures* (groups, rings, fields) as abstract objects, independent of what the elements "are." It lets you do arithmetic on things that are not numbers — like 128-bit blocks.

The **G** in AES-GCM stands for *Galois*, the young French mathematician who founded the theory of finite fields. Your authentication tag is not computed with ordinary multiplication; it is computed by multiplying 128-bit values in the Galois field  $GF(2^{128})$ .

**DEFINITION**

**Galois field  $GF(2^{128})$**  — a finite field whose  $2^{128}$  elements are 128-bit strings interpreted as polynomials with coefficients in  $\{0,1\}$ . You add them by XOR and multiply them modulo a fixed irreducible polynomial, so multiplication never "overflows" the 128 bits.

teaching example

text

```

1 element = 128 bits = a polynomial  b127·x127 + ... + b1·x + b0
2 add(A,B) = A XOR B                  (no carries — it's mod 2)
3 mul(A,B) = (A·B) mod (x128 + x7 + x2 + x + 1) ← GHASH's field

```

This illustrates what AES-GCM's authenticator (GHASH) is doing under the hood: it treats your ciphertext and the AAD as coefficients of a big polynomial and evaluates it in  $GF(2^{128})$ , then encrypts the result into the 16-byte tag. You never wrote this — it is inside `AESGCM(key)` (Python), `Cipher.getInstance("AES/GCM/NoPadding")` (Kotlin), and `AES.GCM` (Swift) — but every `TAG_LEN = 16` byte you put on the wire is an element of that field. This is the discipline of abstract algebra embodied as a 128-bit checksum that is infeasible to forge.

## Probability and one-way functions: SHA-256 and the birthday bound



**DEFINITION**

**Hash function (one-way function)** — a function that maps any input to a fixed-size digest (SHA-256  $\rightarrow$  256 bits) such that it is easy to compute forward but computationally impossible to invert, and impractical to find two inputs with the same digest.

**DEFINITION**

**Birthday bound** — a probability result — in a room of just 23 people two likely share a birthday — that says for an  $n$ -bit hash you expect a collision after only about  $2^{(n/2)}$  random inputs, not  $2^n$ . For SHA-256 that is  $2^{128}$  attempts, still infeasible, which is why 256 bits is the standard.

Your handshake commits to *everything* it has seen by hashing it into one 32-byte value, the transcript hash, using SHA-256. Both sides fold the versions, both public keys, and the pairing id into it:

apps/pc-server/grx\_crypto.py · lines 47-65

Python

```

1 def derive_keys(shared, psk, th):
2     """ECDH shared + pairing PSK + transcript -> per-direction keys + confirm key.
3
4     Each subkey is a single HKDF-SHA256(ikm = shared||psk, salt = transcript,
5     info = label) — one extract+expand per key. This exactly matches Android's
6     Tink Hkdf.computeHkdf("HMACSHA256", ikm, salt, info, L), so both ends derive
7     identical bytes. (Do NOT switch to a master-then-expand scheme: Tink has no
8     standalone HKDF-Expand, so the two languages would diverge.)
9     """
10    ikm = shared + psk
11
12    def hk(label, n):
13        return HKDF(algorithm=hashes.SHA256(), length=n, salt=th, info=label).derive(ikm)
14
15    return {
16        'c2s': hk(b'grx c2s v1', KEY_LEN),
17        's2c': hk(b'grx s2c v1', KEY_LEN),
18        'confirm': hk(b'grx confirm v1', 32),
19    }

```

The `salt=th` argument is the SHA-256 transcript hash. Because a one-way hash has no useful collisions (birthday bound  $2^{128}$ ), an attacker cannot craft a *different* handshake transcript that hashes to the same `th` — so cannot slip a downgraded version or a swapped public key past you unnoticed. The probability discipline shows up as a silent guarantee: the digest is unique-in-practice, therefore the derived keys are bound to exactly the messages that were exchanged.

## Applied cryptography: AEAD, nonces, and the reuse catastrophe

**DEFINITION**

**Applied cryptography** — the engineering sub-discipline that takes proven primitives (ciphers, MACs, key-exchange) and composes them *correctly* into a working protocol — where most real-world breaks happen not in the math but in the composition.

**DEFINITION**

**AEAD (Authenticated Encryption with Associated Data)** — a single primitive that both hides the plaintext (confidentiality) and stamps it with a tag that detects any tampering (integrity), while also authenticating some cleartext "associated data" that travels alongside. AES-128-GCM is your AEAD.

Your plaintext is the same 20-byte input frame the engine already produced (`<Q H B B B B B I` — timestamp, buttons, six axes/triggers, and a now-vestigial `authToken` that GRX sends as 0 because GCM now does the authenticating). GRX seals those 20 bytes into a 41-byte wire frame: `version(1) | counter_low32(4) | ciphertext(20) | tag(16)`.

**DEFINITION**

**Nonce** — a "number used once" — a value fed to the cipher that must be unique for every message encrypted under a given key. GCM's nonce is 96 bits.

Here is the single most important fact in this entire discipline, and your spec states it as invariant #1: *never reuse a (key, nonce) pair in GCM*. It is not a preference; it is a mathematical cliff. Reuse the same nonce with the same key on two messages and GCM leaks the XOR of the two plaintexts **and** — far worse — lets the attacker solve for the GHASH authentication key and forge tags at will. Your code defends this on two independent axes at once:

```
apps/android-client/app/src/main/java/com/gamepad/client/GrxCrypto.kt · lines 113-131 Kotlin

1  class Sender(key: ByteArray, private val direction: Int) {
2      private val secret = SecretKeySpec(key, "AES")
3      private var counter = 0L
4      // Reuse one Cipher instance across seals: Cipher.getInstance does a JCE
5      // provider lookup (allocations + service scan) EVERY call, and this runs on
6      // the hot send path. GCM requires a fresh IV per encryption, which we already
7      // guarantee (counter is strictly monotonic → a distinct nonce every init), so
8      // re-init()ing a cached Cipher is safe and byte-identical on the wire.
9      private val cipher = Cipher.getInstance("AES/GCM/NoPadding")
10     fun seal(plaintext: ByteArray): ByteArray {
11         require(plaintext.size == PLAINTEXT_LEN)
12         counter += 1
13         cipher.init(Cipher.ENCRYPT_MODE, secret, GCMParameterSpec(TAG_LEN * 8, nonce(direction, counter)))
14     }
15     cipher.updateAAD(aad(counter))
16     val body = cipher.doFinal(plaintext) // ciphertext||tag (16-byte tag appended)
17     return ByteBuffer.allocate(WIRE_LEN).order(ByteOrder.LITTLE_ENDIAN)
18         .put(VERSION).putInt(counter.toInt()).put(body).array()
19 }
```

Axis one: **the counter never resets** (`counter += 1`, starting at 1, strictly monotonic, a `Long` that would take centuries to exhaust at 1 kHz). Axis two: **a fresh ephemeral key per connection** — a brand-new

X25519 keypair every time the phone connects, so even a full process restart that lost the counter starts a *new key* rather than replaying an old nonce. `GRX_PROTOCOL.md` makes the dependency explicit: "Fresh ephemeral X25519 keypair each connect (this is what makes counter-from-0 safe; reusing a key with a reset counter is a total GCM break)." The nonce itself is `LE32(direction) || LE64(counter)` — the *direction* byte (1 = phone→PC, 2 = PC→phone) guarantees the two directions never collide even though the counter is per-direction, and each direction gets its own key (`k_c2s`, `k_s2c`) besides.

```
apps/pc-server/grx_crypto.py · lines 123-128 Python
1 def _nonce(direction, counter):
2     return struct.pack('<IQ', direction, counter & 0xFFFFFFFFFFFFFFFF)
3
4
5 def _aad(counter):
6     return struct.pack('<BQ', VERSION, counter & 0xFFFFFFFFFFFFFFFF)
```

The `_aad` (associated data) binds the **version** and **counter** into the authentication without encrypting them, which is your anti-downgrade lever: an attacker cannot rewrite the version byte to force an older, weaker protocol, because that byte is inside what the tag protects. This is applied cryptography in its purest form — the primitives (AES, GHASH) are untouched; the *safety* lives entirely in how you feed them nonces and AAD.

#### IN PLAIN TERMS

Think of GCM as a numbered set of tamper-evident envelopes. The number on each envelope is the nonce. The one rule the factory ships with: never lick two different letters into envelopes with the same number. If you do, a clever bystander can hold the two up to the light, line up the numbers, and reconstruct both letters *and* learn to counterfeit your wax seal. Your fix is to give every phone connection a fresh box of envelopes (new key) and to never, ever restart the numbering (never-reset counter).

## Key derivation and the QR binding that kills MITM

#### DEFINITION

**HKDF / HMAC** — HKDF is a key-derivation function built from HMAC (a keyed hash). It "extracts" concentrated randomness from a raw secret and "expands" it into as many independent, fixed-length keys as you need — here, one for each direction plus a confirmation key.

You do not use the raw X25519 output as a key directly. You run it through HKDF-SHA256 with the transcript as salt and a distinct label per output (`grx c2s v1`, `grx s2c v1`, `grx confirm v1`), giving three keys that are cryptographically independent even though they share one seed. But the seed is the subtle part:

**DEFINITION**

**MITM (man-in-the-middle)** — an attacker who sits between phone and PC, relaying and possibly altering messages, trying to impersonate each side to the other and establish two separate encrypted links it can read.

X25519 alone stops eavesdropping but *not* an active MITM — the attacker could run its own key exchange with each side. What stops that is folding a shared secret the attacker does not have into the key derivation. That secret is the **PSK**, derived from the pairing key already inside your QR code:

```
apps/pc-server/server.py · lines 1286-1295 Python
1 def _grx_psk_from_key(key_str):
2     """32-byte GRX PSK derived from the shared pairing key (HKDF-SHA256). Both
3     ends already hold `key` (it's in the QR), so this needs no new pairing data."""
4     from cryptography.hazmat.primitives.kdf.hkdf import HKDF
5     from cryptography.hazmat.primitives import hashes
6     try:
7         ikm = bytes.fromhex(key_str)
8     except (ValueError, TypeError):
9         ikm = (key_str or "").encode()
10    return HKDF(algorithm=hashes.SHA256(), length=32, salt=b"", info=b"grx psk v1").derive(ikm)
```

Because `derive_keys` mixes `ikm = shared + psk`, an attacker who completes the X25519 exchange but lacks the PSK derives *different* keys — and then fails the confirmation step. Each side sends `HMAC-SHA256(k_conf, transcript || role)` and refuses to accept any input until the other's confirm verifies. In Swift you can watch the MITM die at that exact **guard**:

```
apps/ios-client/Sources/GrxSession.swift · lines 78-89 Swift
1 // Flat single-shot HKDFs (salt = transcript hash) — NOT the two-stage
2 // "grx master v1" scheme the protocol doc describes; the shipped
3 // Kotlin/Python code is normative here.
4 let ikm = sharedBytes + psk
5 let c2s = Self.hkdf(ikm: ikm, salt: th, info: Data("grx c2s v1".utf8), length: 16)
6 let s2c = Self.hkdf(ikm: ikm, salt: th, info: Data("grx s2c v1".utf8), length: 16)
7 let conf = SymmetricKey(data: Self.hkdf(ikm: ikm, salt: th, info: Data("grx confirm v1".utf8), leng
th: 32))
8
9 // Constant-time verify of HMAC-SHA256(k_conf, th || 'S').
10 guard HMAC<SHA256>.isValidAuthenticationCode(serverConfirm, authenticating: th + Data([0x53]), usin
g: conf) else {
11     return true // wrong pairing key / MITM: stay unestablished, send nothing
12 }
```

Two disciplines appear here at once. First, applied cryptography: `isValidAuthenticationCode` is a *constant-time* comparison — it takes the same time whether the tag is wrong in the first byte or the last, denying an attacker the timing side-channel that a naive `==` would leak. Second, an honest note on your own docs: the comment records that the shipped code uses **flat single-shot HKDFs**, *not* the two-stage "grx master v1" scheme `GRX_PROTOCOL.md` step 4 still describes. The code wins. The Python docstring above explains why — Android's Tink library has no standalone HKDF-Expand, so a master-then-expand design would make the two languages derive different bytes and never interoperate. This is exactly the applied-crypto lesson that the *implementable* design beats the *elegant* one.

#### WATCH OUT

The PSK's real entropy is only as large as the QR pairing key, which [ARCHITECTURE.md](#) describes as a random 4-byte hex value — 32 bits. HKDF stretches it to 32 bytes but *cannot manufacture entropy*. So the MITM-resistance argument is mathematically sound but quantitatively bounded: an attacker who captures one handshake could, in principle, brute-force a 32-bit key offline ( $2^{32}$  candidate PSKs, checking each against the captured confirm MAC). The discipline's honest verdict is that your QR channel, not the crypto, is the entropy floor — a longer pairing key would raise it cheaply.

## Security engineering: threat model, replay window, and the hole GRX closed

#### DEFINITION

**Threat model** — an explicit statement of *who* you are defending against and *what* they can do. Without one, "secure" is meaningless. Yours is stated in [GRX\\_PROTOCOL.md](#): "On-path attacker on the same LAN or USB host," who must be denied confidentiality, integrity, replay, and downgrade.

#### DEFINITION

**Replay attack** — capturing a legitimate encrypted packet and re-sending it later. GCM's tag would still verify — the packet is authentic — so AEAD alone does not stop replay. You need a separate mechanism.

That mechanism is the sliding replay window, and its *ordering* is the whole game:

```
apps/pc-server/grx_crypto.py · lines 146-168 Python

1 class GrxReceiver:
2     def __init__(self, key, direction, window_bits=1024):
3         self.aead = AESGCM(key)
4         self.direction = direction
5         self.win = ReplayWindow(window_bits)
6
7     def open(self, frame):
8         """Returns the 20-byte plaintext, or None to DROP (old/dup/forged/garbage)."""
9         if len(frame) != WIRE_LEN:
10             return None
11         ver, low32 = struct.unpack('<BI', frame[:HEADER_LEN])
12         if ver != VERSION:
13             return None
14         counter = self.win.reconstruct(low32)
15         if not self.win.check(counter):
16             return None # 2. early dup/old reject
17         try: # 3. verify tag FIRST
18             pt = self.aead.decrypt(_nonce(self.direction, counter),
19                                   frame[HEADER_LEN:], _aad(counter))
20         except Exception:
```

```
apps/pc-server/grx_crypto.py · (cont.) Python
21         return None                # auth fail -> drop, window UNTOUCHED
22         self.win.commit(counter)    # 4. advance ONLY on auth success
23         return pt
```

Read the numbered comments as a security proof. The window is 1024 packets wide so out-of-order UDP (a real fact of your loss-tolerant transport) is tolerated. A duplicate or too-old counter is rejected *before* spending a decrypt (`check`). Then — and this is invariant #2 — the tag is verified **before** the window is ever advanced. Only on authentic success does `commit` slide the high-water mark. If you had advanced the window first, an attacker could forge a packet with a huge counter (`0x7FFFFFFF`), fail authentication, but *still* push the window forward — and every real packet behind it would now look "too old" and be dropped: an input blackout. Your own self-test asserts precisely this (`rcv3.win.high == 0` after a forged high-counter packet). Only 33 bytes of receiver logic, but it encodes the anti-DoS invariant that most hand-rolled protocols get wrong. The Kotlin `Receiver.open` and Swift `open` mirror it bit-for-bit with a 64-entry window for the low-rate reverse channel.

#### NOTE

The reverse (server→client) direction is fully implemented but, per the Swift comment, "dormant in v1 — server rumble is still cleartext." So the security discipline is present but exercised only lightly on the rumble path today; the machinery is ready the moment you encrypt rumble.

Finally, security engineering is also about *what you removed*. GRX exists to retire a live hole. The pre-GRX design authenticated packets with a 32-bit `authToken` sent in the clear on every frame:

```
apps/docs/ARCHITECTURE.md · lines 240-241 text
1  4. `key` is parsed as a hex integer and written to `authToken` field in every packet (`uint32`).
2  5. Server validates `authToken` on every received packet (ignores packets with wrong token).
```

That is the anti-pattern GRX replaces, and naming it precisely is the point of a threat model. An on-path attacker who sniffed a single packet learned the token forever — then could forge inputs, replay them, or spoof the phone entirely, because the "secret" was broadcast in cleartext on every send. `GRX_PROTOCOL.md` opens by calling it exactly that: "the current cleartext, replayable 32-bit `authToken` (a live security hole)." The token now travels as a hardcoded 0 — GCM's 128-bit tag does the authenticating instead.

#### WATCH OUT

The hole is *closed by design* but not yet *bolted shut in production*. `server.py` still runs with `GRX_REQUIRED = False`, meaning the legacy cleartext path is accepted in parallel until the flag is flipped. A downgrade attacker could, today, simply speak the old 20-byte cleartext dialect and be served. The comment beside the flag says it plainly: flip it True "once the Android GRX build ships → drops the legacy cleartext path (closes the `authToken` downgrade hole)." The cryptography is finished; the security-engineering rollout is one boolean from complete.

---

DISCIPLINE 7

# Operating Systems & Systems Programming

7

## The Discipline, and Where It Hides

Operating-systems theory studies how software shares one machine's hardware — its CPUs, memory, devices, and time — safely and efficiently. Systems programming is the craft of writing code that works directly with those OS facilities: threads, locks, drivers, file descriptors, and processes. GamepadOS lives close to the metal on both ends (a real-time thread on the phone, a driver-backed virtual device on the PC), so OS concepts are everywhere in its guts.

### DEFINITION

**Operating system (OS)** — the program that manages a computer's hardware and shares it among many programs — scheduling the CPU, handing out memory, and mediating access to devices.

### DEFINITION

**Concurrency** — multiple threads of execution making progress at once. It brings speed and responsiveness but also *race conditions* — bugs where the outcome depends on unlucky timing.

### DEFINITION

**Device driver** — OS code that lets programs talk to a piece of hardware through a uniform interface. A *virtual* driver presents fake hardware that the OS treats as real.

## Concurrency — Threads, Atomics, and Races

The engine and the server both do several things at once, which means **threads** and the discipline of sharing data between them safely. The C++ engine has a dedicated send thread reading state that other threads write, and it uses **atomic** variables so those reads and writes can't tear or race:

```
apps/android-client/app/src/main/cpp/gamepad-engine.cpp · lines 118-130 C++

1 // ONLY reliable "the PC is actually there" signal: UDP sendto() succeeds locally
2 // even when nothing is listening, so a rising packet-send count does NOT mean
3 // connected. The link is "alive" only if an ACK arrived recently.
4 std::atomic<uint64_t> lastAckMonoNs(0);
5
6 // Rumble from the PC over Wi-Fi (UDP). The PC sends "RMB" + large + small for
7 // THIS phone's virtual pad. We stash the latest motor values and bump a sequence
8 // so the JS layer can poll, apply the user's on/off + intensity, and fire the
9 // vibrator — mirroring the USB/WebSocket rumble path (which the Wi-Fi path was
10 // missing entirely, so wireless phones never felt in-game rumble).
11 std::atomic<int>      rumbleLeft(0);    // large / low-freq motor, 0..255
12 std::atomic<int>      rumbleRight(0);   // small / high-freq motor, 0..255
13 std::atomic<uint32_t> rumbleSeq(0);     // ++ on every RMB datagram received
```



**DEFINITION**

**Atomic variable** — a value the CPU reads and writes in one indivisible step, so a concurrent reader never sees a half-updated value ("torn read"). Atomics are the lightest-weight tool for safe sharing between threads.

Here the receive thread writes `rumbleLeft/rumbleRight` whenever a rumble datagram arrives, while the telemetry code reads them to report to the UI — different threads touching the same memory. Marking them `std::atomic` guarantees each read sees a whole value, never a half-written one (a "torn read"), and `lastAckMonoNs` is atomic for the same reason: the send loop reads it to decide "am I connected?" while the receive path writes it on every ACK. Protecting a value that two threads share is race-condition reasoning — the core discipline of concurrent systems programming. On the PC side, the server guards its virtual pads with a `GAMEPAD_LOCK` **mutex** so that only one thread mutates a pad at a time — the heavier-weight sibling of the atomic, used when a whole multi-step update must be indivisible.

## The Kernel/Driver Boundary — Faking Hardware

The most striking OS concept in GamepadOS is how the PC server makes a *fake controller Windows believes is real*. It does this through **ViGEmBus**, a kernel-mode virtual bus driver, via the `vgamepad` library:

apps/pc-server/server.py · lines 834-841

Python

```

1         try:
2             with GAMEPAD_LOCK:
3                 pad = vg.VX360Gamepad()
4                 reset_gamepad_state(pad)
5                 pad.update()
6         except Exception:
7             return None
8         s = PadSession(pad)

```

**DEFINITION**

**Kernel space vs user space** — the kernel is the privileged core of the OS that talks to hardware; user space is where ordinary programs run, walled off from the hardware for safety. A driver lives in kernel space; your program asks it to act on your behalf.

`vg.VX360Gamepad()` asks the ViGEmBus kernel driver to **plug in a virtual Xbox 360 controller**. From that instant, Windows' input subsystem — and therefore every game — sees a real XInput device on one of its four controller slots. Your Python program runs in ordinary user space and could never pretend to be hardware on its own; it's the kernel driver that bridges the gap, exactly the user/kernel division OS theory describes. The USB path uses the same idea from the other direction: **WinUSB** is the driver that lets the AOA transport claim the phone's bulk endpoints.

#### DEFINITION

**XInput slot** — Windows exposes up to four controller "slots." Registering a virtual pad occupies one, which is why the server caps concurrent phones at four — an OS-imposed resource limit the code respects.

## Processes, File Descriptors, and Packaging

Systems programming means working with the OS's low-level handles. The AOA transport receives the phone connection as a raw **file descriptor** passed down from Kotlin (`detachFd`) into C++, where the engine reads and writes it like any other stream.

#### DEFINITION

**File descriptor (fd)** — a small integer the OS hands you as a handle to an open resource — a file, a socket, or here a USB accessory connection. To the program, everything is "just an fd" you read and write.

The PC server also grapples with **process** and **filesystem** realities when it ships as a frozen executable. PyInstaller unpacks the app into a temporary folder at launch (`sys._MEIPASS`), and the server carefully copies bundled tools *out* of that temp dir so a background process doesn't hold the files open and block cleanup on exit:

```
apps/pc-server/server.py · lines 1018-1024 Python
1  meipass = getattr(sys, "_MEIPASS", None)
2  if not meipass:
3      return None
4  try:
5      import shutil
6      dst_dir = os.path.join(_gp_config_dir(), "adb") # %LOCALAPPDATA%\GamepadServer\adb
7      os.makedirs(dst_dir, exist_ok=True)
```

`sys._MEIPASS` is the temp directory the frozen exe extracts itself into — a detail you only meet when you understand that a "single .exe" is really a self-unpacking archive that becomes a running **process** with open file handles. Managing where those files live, and which process holds them, so that shutdown is clean, is exactly the kind of OS-level housekeeping systems programmers do. Installing the update, in turn, requires **privilege elevation** (Windows UAC) because writing to **Program Files** needs kernel-granted rights — another place the OS's permission model shapes the code.

## Language Runtimes and the JNI Boundary

Finally, OS theory touches *why the project mixes languages*. The Kotlin shell runs in a **managed runtime** with **garbage collection** (the OS-scheduled runtime periodically pauses to reclaim memory); the C++ engine runs as **native code** with no such pauses. The two meet across the **JNI** boundary — the OS-level bridge between the managed and native worlds — with function names like `Java_com_gamepad_client_MainActivity_initNetworkNative`. The whole reason the hot path is C++ is an OS-scheduling fact: a garbage-collector pause is an uncontrollable stall, and a real-time send loop

cannot tolerate one. Choosing native code to escape GC pauses is an operating-systems argument, not a language-preference one.

## Where Each Branch Lives

| OS / systems sub-field                | Where in GamepadOS                                               |
|---------------------------------------|------------------------------------------------------------------|
| Concurrency (threads, atomics, mutex) | <code>std::atomic</code> engine state, <code>GAMEPAD_LOCK</code> |
| Kernel/driver boundary                | ViGEmBus virtual X360 pad, WinUSB for AOA                        |
| Resource limits                       | the 4-slot XInput cap                                            |
| File descriptors                      | the AOA accessory fd from <code>detachFd</code>                  |
| Processes & packaging                 | PyInstaller <code>_MEIPASS</code> , UAC elevation                |
| Runtimes & the native boundary        | JNI, avoiding GC pauses on the hot path                          |

From a real-time thread fighting the scheduler for punctuality, to a kernel driver conjuring a controller out of nothing, to a frozen process minding its own file handles — GamepadOS is, under the hood, a sustained exercise in operating-systems and systems-programming craft.

---

DISCIPLINE 8

# Information & Coding Theory

8

Every part of GamepadOS that turns a thing into *bytes* — a button press, a random key, a downloaded installer, the little black-and-white square on your PC screen — is quietly practising a discipline that was invented in 1948 by an engineer named Claude Shannon.

**DEFINITION**

**Information theory** — the mathematics of *how much* information a message actually carries, measured in bits, and of the fundamental limits on storing or transmitting it over an imperfect channel. It answers questions like "how small can this get?" and "how much noise can this survive?".

**DEFINITION**

**Coding theory** — the sister discipline of *how* to represent information as symbols — one branch (source coding) squeezes messages smaller, another (channel coding) adds carefully-chosen redundancy so damage can be detected or repaired.

The two split into a handful of sub-branches, and your project touches all of them:

| Sub-branch                       | Where it lives in GamepadOS                                                                  |
|----------------------------------|----------------------------------------------------------------------------------------------|
| Source coding (compact encoding) | The 20-byte <code>GamepadPayload</code> struct; <code>&lt;Q H B B B B B B I</code>           |
| Entropy / randomness             | <code>secrets.token_hex(4)</code> , the GRX PSK, X25519 ephemeral keys                       |
| Error detection                  | SHA-256 + <code>MZ</code> magic header in <code>download_update</code> ; the 128-bit GCM tag |
| Error correction                 | The QR code's Reed-Solomon; <code>CHANGED_PACKET_REDUNDANCY</code>                           |

**IN PLAIN TERMS**

Think of shipping a fragile vase. Source coding is packing it into the *smallest* box that still holds it. Channel coding is the *bubble wrap* — extra material that carries no vase but lets the vase survive a rough courier. Information theory is the physics that tells you the minimum box size and how much bubble wrap the road actually demands. GamepadOS makes a different bubble-wrap decision on each of its channels, and every decision is a theorem in disguise.

## Encoding and Compactness — your 20-byte frame is source coding

**DEFINITION**

**Source coding (data compaction)** — choosing a representation that uses as few bits as possible to carry the message. A *fixed-length* code assigns every message the same number of bits; the win comes from choosing the tightest layout, not from variable-length tricks.

The single most important design choice in the whole system is that a controller frame is exactly twenty bytes with no wasted bit. It is declared once in C++ as a packed struct:

```
apps/android-client/app/src/main/cpp/gamepad-engine.cpp · lines 51-65 C++

1  #pragma pack(push, 1)
2  struct GamepadPayload {
3      uint64_t timestamp; // 8 bytes
4      uint16_t buttons; // 2 bytes
5      uint8_t leftTrigger; // 1 byte
6      uint8_t rightTrigger; // 1 byte
7      uint8_t leftStickX; // 1 byte
8      uint8_t leftStickY; // 1 byte
9      uint8_t rightStickX; // 1 byte
10     uint8_t rightStickY; // 1 byte
11     uint32_t authToken; // 4 bytes
12 };
13 #pragma pack(pop)
14
15 static_assert(sizeof(GamepadPayload) == 20, "FATAL: Payload struct is not exactly 20 bytes!");
```

This is a fixed-length source code, hand-designed. `#pragma pack(1)` strips the padding a compiler would normally insert, and the `static_assert` refuses to build if the layout ever drifts off 20 bytes — a compile-time guarantee that the encoding contract cannot silently rot. Contrast this with the obvious alternative, JSON: `{"buttons":5,"leftTrigger":255,...}` is roughly 120–150 characters, six to seven times larger, and it must be *parsed* (scanning for quotes and commas) rather than read straight out of fixed offsets. At your target of up to 1000 packets per second, that size and parse cost is not academic. A fixed binary layout is decoded by pointing at byte 12 and reading it; there is no ambiguity and no scanning. Compactness here buys latency, not disk.

The receiver's half of the contract is one line of Python:

```
apps/pc-server/server.py · lines 124-125 Python

1  PAYLOAD_FORMAT = '<Q H B B B B B I'
2  PAYLOAD_SIZE = struct.calcsize(PAYLOAD_FORMAT)
```

Each letter is a field width — `Q`=8-byte unsigned, `H`=2-byte, `B`=1-byte, `I`=4-byte — and `struct.calcsize` computes 20 from them, so the same numbers exist in exactly one place on each side. The leading `<` is a piece of coding theory people forget is a *choice*.

#### DEFINITION

**Endianness** — the convention for which end of a multi-byte number comes first in memory. Little-endian ("`<`") puts the least-significant byte first; big-endian puts the most-significant byte first.

Endianness is a *positional* encoding decision: the bytes `01 00` mean the number 1 under little-endian and the number 256 under big-endian. If the two ends disagreed, every timestamp and auth token would be silently scrambled. The `<` pins both sides to little-endian, which is why the C++ engine ships `htole64/htole32` helpers (lines 26–28) that are deliberately identity operations — Android's ARM is already little-endian, so the swap is free, but the intent is documented so a big-endian port could not forget it.

**WATCH OUT**

The code comment above the format string (server.py line 114) still calls this a "16-Byte Payload Structure." That label is stale: it predates the 4-byte `authToken`, and `PAYLOAD_SIZE = struct.calcsize(...)` genuinely evaluates to 20. When a comment and the computed value disagree, trust the computed value — the wire is 20 bytes.

The densest corner of the frame is the `buttons` field: two bytes, sixteen bits, into which fifteen distinct controls are packed one-per-bit. The PC unpacks them by masking:

```
apps/pc-server/server.py · lines 986-1000 Python
1      XB = vg.XUSB_BUTTON
2      if buttons & (1 << 0): gamepad.press_button(button=XB.XUSB_GAMEPAD_A)
3      if buttons & (1 << 1): gamepad.press_button(button=XB.XUSB_GAMEPAD_B)
4      if buttons & (1 << 2): gamepad.press_button(button=XB.XUSB_GAMEPAD_X)
5      if buttons & (1 << 3): gamepad.press_button(button=XB.XUSB_GAMEPAD_Y)
6      if buttons & (1 << 4): gamepad.press_button(button=XB.XUSB_GAMEPAD_LEFT_SHOULDER)
7      if buttons & (1 << 5): gamepad.press_button(button=XB.XUSB_GAMEPAD_RIGHT_SHOULDER)
8      if buttons & (1 << 6): gamepad.press_button(button=XB.XUSB_GAMEPAD_START)
9      if buttons & (1 << 7): gamepad.press_button(button=XB.XUSB_GAMEPAD_BACK)
10     if buttons & (1 << 8): gamepad.press_button(button=XB.XUSB_GAMEPAD_LEFT_THUMB)
11     if buttons & (1 << 9): gamepad.press_button(button=XB.XUSB_GAMEPAD_RIGHT_THUMB)
12     if buttons & (1 << 10): gamepad.press_button(button=XB.XUSB_GAMEPAD_DPAD_UP)
13     if buttons & (1 << 11): gamepad.press_button(button=XB.XUSB_GAMEPAD_DPAD_DOWN)
14     if buttons & (1 << 12): gamepad.press_button(button=XB.XUSB_GAMEPAD_DPAD_LEFT)
15     if buttons & (1 << 13): gamepad.press_button(button=XB.XUSB_GAMEPAD_DPAD_RIGHT)
```

Bits 0 through 13 carry fourteen buttons, and `apply_inputs` adds bit 14 (the Guide/Xbox button) just below, for fifteen booleans in sixteen bits. This is *information density* in its purest form: a boolean is one bit of information, and you store it as one bit, not as a whole byte or a JSON `true`. The naive layout — one byte per button — would need fifteen bytes; the bit-packed layout needs two, a better-than-7× win on the busiest field. `(1 << n)` builds the mask for bit `n`, and `buttons & mask` tests it. One spare bit (bit 15) is your headroom for a sixteenth control at zero extra cost.

## Entropy and Keys — measuring unpredictability

**DEFINITION**

**Entropy** — the amount of genuine unpredictability in a value, measured in bits. A value with  $n$  bits of entropy is one an attacker must, on average, guess among roughly  $2^n$  equally-likely possibilities. Crucially, entropy is a property of *how the value was generated*, not of how many bytes it occupies.

Your pairing key is born from a cryptographically secure random source:

apps/pc-server/server.py · lines 1908-1913

Python

```

1         if len(k) == 8:
2             int(k, 16) # validates hex
3             return k
4         except Exception:
5             pass
6         k = secrets.token_hex(4)

```

`secrets.token_hex(4)` draws **4 bytes** from the operating system's CSPRNG and renders them as 8 hex characters. Four bytes is 32 bits of entropy: an attacker faces about 4.3 billion equally-likely keys. The `secrets` module (not `random`) matters — `random` is a predictable pseudo-random generator seeded from the clock, whereas `secrets` pulls from the OS entropy pool (`os.urandom`), the same tap used for real cryptographic keys. That is the "randomness source" question answered correctly.

That 4-byte key is then stretched into the GRX pre-shared key:

apps/pc-server/server.py · lines 1286-1295

Python

```

1 def _grx_psk_from_key(key_str):
2     """32-byte GRX PSK derived from the shared pairing key (HKDF-SHA256). Both
3     ends already hold `key` (it's in the QR), so this needs no new pairing data."""
4     from cryptography.hazmat.primitives.kdf.hkdf import HKDF
5     from cryptography.hazmat.primitives import hashes
6     try:
7         ikm = bytes.fromhex(key_str)
8     except (ValueError, TypeError):
9         ikm = (key_str or "").encode()
10    return HKDF(algorithm=hashes.SHA256(), length=32, salt=b"", info=b"grx psk v1").derive(ikm)

```

HKDF turns the 4-byte input into a **32-byte** (256-bit-*long*) PSK. This is where information theory gives a sharp, humbling warning.

#### WATCH OUT

Length is not entropy. A key-derivation function cannot manufacture unpredictability it wasn't given. The PSK is 256 bits *long*, but its input (`ikm`) is the 4-byte pairing key, so it carries only ~32 bits of true entropy. `GRX_PROTOCOL.md` describes the QR as carrying a genuine "32-byte pre-shared key" (256 bits of secret); the shipped code carries 4 random bytes and expands them. The code is the source of truth — the *authentication* strength that resists a man-in-the-middle is bounded by  $\sim 2^{32}$ , not  $2^{256}$ , even though the bytes on the wire look 256-bit strong.

#### NOTE

The session's *confidentiality* rests on something stronger. The GRX handshake generates a fresh ephemeral X25519 keypair on every connection (`GRX_PROTOCOL.md` line 26), so the per-message encryption keys come from a full elliptic-curve exchange (~128-bit security), not from the 32-bit pairing key. The pairing key's job is only to authenticate that handshake. So the honest summary is: strong per-session secrecy, but a pairing/authentication entropy margin smaller than the PSK's byte-count advertises. Widening `token_hex(4)` to `token_hex(16)` would close that gap the moment the QR and phone can carry the longer key.



**EXPERT DETAIL**

Why 32 bits is defensible *here*: the pairing key authenticates a 1-RTT handshake against an *on-path LAN attacker* (GRX's stated threat model), and the never-reset GCM counter plus per-connection ephemeral keys mean there is no offline oracle to hammer  $2^{32}$  against at line rate. The margin is thin, not broken — but it is a length-vs-entropy trap worth naming precisely.

## Error Detection — catching a corrupt message

**DEFINITION**

**Error detection** — adding redundant check-bits so that a receiver can tell whether a message arrived intact, without necessarily being able to repair it. A checksum and a "magic number" are two classic detectors.

Your in-app updater refuses to run a download it cannot vouch for, using two independent detectors:

```
apps/pc-server/server.py · lines 236-246 Python
1         if done == 0:
2             return (False, "downloaded 0 bytes")
3         if first[:2] != b"MZ":
4             return (False, "not a valid Windows installer")
5         if expected_sha256 and h.hexdigest().lower() != str(expected_sha256).lower():
6             try:
7                 os.remove(dest_path)
8             except Exception:
9                 pass
10            return (False, "checksum mismatch – download corrupt")
11        return (True, None)
```

The **MZ** check is a *magic number*: every genuine Windows executable begins with the two bytes **M**, **Z** (the initials of DOS architect Mark Zbikowski). If a captive-portal or a 404 handed you an HTML error page instead of an installer, its first bytes are **<!** or **<h**, and this two-byte test catches it instantly — a cheap, coarse detector. The SHA-256 comparison is the fine one.

**DEFINITION**

**Cryptographic hash / checksum** — a fixed-size fingerprint (256 bits for SHA-256) computed from every byte of a file; flipping even one input bit changes the fingerprint unrecognisably, so a matching hash is overwhelming evidence the bytes are exactly the intended ones.

Because the hash was accumulated incrementally as the file streamed (`h.update(chunk)`), a single corrupted or truncated byte anywhere makes `hexdigest()` differ from the manifest's `expected_sha256`, the partial file is deleted, and nothing runs. That is textbook end-to-end integrity: detect, discard, never execute a damaged message.

**NOTE**

The same idea guards the *input* path. GRX's per-packet AES-128-GCM appends a full 128-bit authentication tag (GRX\_PROTOCOL.md line 52); any bit-flip or forgery makes the tag verification fail and the packet is dropped (protocol step 3). A GCM tag is error *detection* fused with authentication — it catches accidental corruption and deliberate tampering with the same 16 bytes.

## Error Correction — surviving damage, not just spotting it

**DEFINITION**

**Error-correcting code (ECC)** — redundancy chosen so the receiver can not only detect damage but *reconstruct* the original from what survived. Reed-Solomon is the family used by CDs, DVDs, and QR codes.

Your pairing QR is a Reed-Solomon-protected message:

apps/pc-server/server.py · lines 1926-1930

Python

```
1 csv_str = f"{ip},{port},{key}"
2 qr = qrcode.QRCode(box_size=6, border=2)
3 qr.add_data(csv_str)
4 qr.make(fit=True)
5 qr_pil = qr.make_image(fill_color="black", back_color="white").get_image()
```

The payload is a tiny CSV: *ip,port,key*. The QR standard wraps that in Reed-Solomon check symbols so the code still decodes when part of it is smudged, glare-washed, or covered by a finger. With the library's default error-correction level (M, ~15% of the modules recoverable), roughly a seventh of the square can be destroyed and your phone still reads the right IP and key. The two tuning constants matter to that survival: *box\_size=6* renders each module as a 6×6 pixel block so the phone camera resolves modules cleanly, and *border=2* is the *quiet zone* — two modules of mandatory white margin so the scanner can find the code's edges. *fit=True* picks the smallest QR *version* that holds the data, keeping the modules as large (and as robust) as possible. This is the one place in GamepadOS that does true forward error correction: the redundancy travels with the message and repairs it on arrival.

There is a second, subtler correcting code on the hot path — a *repetition code*:

apps/android-client/app/src/main/cpp/gamepad-engine.cpp · lines 192-198

C++

```
1 // Cheap loss-recovery redundancy: when the input CHANGES, send the SAME packet
2 // this many times back-to-back over consecutive polls. The PC server dedups by
3 // content + orders by timestamp, so duplicates are harmless no-ops; this just
4 // makes a single dropped UDP datagram recover within a couple of 2ms polls
5 // instead of waiting up to ~33ms for the next change or keep-alive. Keep-alive
6 // (heartbeat) packets are NOT duplicated.
7 static const int CHANGED_PACKET_REDUNDANCY = 3;
```

When an input changes, the engine sends the identical frame three times. That is the simplest error-correcting code that exists: a repetition code. It trades bandwidth for reliability so that one dropped

UDP datagram doesn't cost you a missed jump. And it embodies a deliberate coding-theory *choice between two families* — forward error correction (send redundancy up front) versus ARQ (detect a loss and ask for a resend). ARQ would cost a full round-trip of latency per loss; on a real-time input stream where a late packet is worse than a lost one, you correctly pick FEC-by-repetition. The server's content-dedup (a duplicate is a no-op) makes the extra copies free of side effects.

## The road not taken — deliberately no compression

The final information-theory decision is one you can only see by its *absence*: nowhere on the input path is a frame compressed. That is correct, and for two reasons rooted in the theory. First, 20 bytes is already far below any network MTU, so a compressor could not split the packet into fewer round-trips — the transmission-time floor is set by the network, not the byte count. Second, compression spends CPU time and adds an encode/decode step to every one of up to 1000 packets per second, injecting exactly the latency and jitter the whole C++ engine exists to eliminate. Shannon's source-coding theorem says compression pays when a channel is *bandwidth*-limited; your channel is *latency*-limited, so the right move is the compact fixed layout of §1 plus the redundancy of §4 — spend bits to beat loss, never spend time to save bits.

### EXPERT DETAIL

The frame is also close to incompressible in practice. Sticks and triggers are near-uniform 8-bit values and the timestamp is monotonically increasing, so the 20-byte frame carries close to 20 bytes of entropy — there is little statistical redundancy for a compressor to remove even if latency were free. The one field with structure, **buttons**, is already bit-packed by hand, which is the optimal fixed-length code for it. GamepadOS thus sits at a tidy information-theoretic sweet spot: maximally dense where density is free (bit-packing), redundant only where loss is the enemy (QR Reed-Solomon, packet repetition), and never compressed where the clock is the enemy.

---

DISCIPLINE 9

# Human-Computer Interaction & Perception

9

## The disciplines, named from scratch

Everything in GamepadOS ends at a human: a thumb on glass, an eye on a screen, a palm feeling a buzz. The field that studies that meeting point has a name, and it splits into several sub-branches that your code quietly obeys.

### DEFINITION

**Human-Computer Interaction (HCI)** — The study of how people use computers and how to design systems so that use is fast, accurate, and comfortable. It sits between computer science, psychology, and design.

### DEFINITION

**Psychophysics** — The branch of perceptual psychology that measures the *quantitative* relationship between a physical stimulus (milliseconds of delay, millimetres of vibration, degrees of tilt) and the sensation a human actually notices. It asks "how big must a change be before a person feels it?"

### DEFINITION

**Haptics** — The science and engineering of touch feedback — delivering information to a user through vibration, force, or texture rather than through light or sound.

### DEFINITION

**Ergonomics** — The study of fitting a tool to the human body and its limits — reach, grip, muscle memory, error tolerance.

### DEFINITION

**Interaction design** — The craft of shaping *what a control does when you touch it and how it tells you it worked* — mappings, feedback, affordances, and the visual language that carries them.

The through-line of this whole section: each of these is a real academic discipline, and each already lives, by name or by behaviour, inside `App.tsx`, `MainActivity.kt`, and your Widgets. Here is where, and why.

| Sub-branch                         | Where it lives in GamepadOS                                                                                  |
|------------------------------------|--------------------------------------------------------------------------------------------------------------|
| Latency perception / psychophysics | <code>useHeld</code> ref-then-send, 16 ms heartbeat, 8 ms gyro data loop                                     |
| Haptics & multimodal feedback      | <code>doVibrate</code> PWM duty-cycle, <code>getHapticCapabilities</code> tiers, <code>onRumblePacket</code> |
| Ergonomics & embodiment            | circular deadzone, gyro tilt-steering, <code>CALIBRATE</code> , per-finger pointer ownership                 |

Visual &amp; interaction design

dark SVG widgets, opacity, the 6.5 px gyro top-edge indicator, z-order

## Latency perception and the psychophysics of milliseconds

A human hand-eye loop is slow — roughly 150–250 ms from "I decide to press" to "I see the result." But that is the *whole* loop; the part your software controls is the *added* delay, and here the psychophysics is brutal. Skilled players reliably detect input latency in the 30–50 ms range, and every extra frame degrades their scores. This is a **just-noticeable-difference** problem.

### DEFINITION

**Just-noticeable-difference (JND)** — The smallest change in a stimulus that a person can reliably perceive. Below the JND, a change is invisible; above it, it feels wrong.

### DEFINITION

**Frame budget** — On a 60 Hz display each frame lasts about 16.7 ms. Anything that misses its frame is not shown until the next one, so 16.7 ms is the atomic unit of visual delay on the phone.

Your central design decision — call it **ref-then-send, state-last** — is a direct exploitation of the fact that the *game* and the *on-screen highlight* have completely different latency budgets. The game must receive the input immediately; the coloured glow under the thumb can safely wait one frame, because the eye will not notice a 16 ms delay on its own finger. `useHeld` encodes exactly that priority order.

apps/controller-ui/src/app/App.tsx · lines 211-220

lang

```

1  const dn = useCallback((id: BtnId) => {
2    heldRef.current = new Set([...heldRef.current, id]);
3    if (onChangeRef.current) onChangeRef.current();
4    setHeld(new Set(heldRef.current));
5  }, []);
6  const up = useCallback((id: BtnId) => {
7    const n = new Set(heldRef.current); n.delete(id); heldRef.current = n;
8    if (onChangeRef.current) onChangeRef.current();
9    setHeld(new Set(heldRef.current));
10 }, []);

```

Read the three lines of `dn` in order. First the *ref* (`heldRef.current`) is mutated — a synchronous, render-free write. Second, `onChange()` fires, which builds and transmits the 20-byte packet from that live ref. Only *third* does `setHeld` schedule a React re-render, which repaints the button. The packet leaves before a single pixel changes. That ordering is not an accident of style; the comment above the hook calls it "the same ref-then-send design the analog sticks use," and the sticks, triggers, and gyro all repeat it. In psychophysics terms: the sub-JND channel (the game) is served first, the supra-JND-tolerant channel (the visual highlight) is served last.

**IN PLAIN TERMS**

Imagine a waiter who takes your order to the kitchen *before* writing it on his notepad. The kitchen (the game) starts cooking instantly; the notepad (the screen) catches up a moment later. You never notice the notepad lag, but you'd absolutely notice slow food. That reversed order is the whole trick.

The same budget-thinking splits time in two other places. The gyroscope runs **two loops on purpose**: a 120 Hz data loop (an 8 ms `setInterval`) that reads the sensor and fires the packet, and a separate 60 Hz `requestAnimationFrame` loop that only slides the on-screen bar. The comment is explicit that this exists so "the game gets ~4 ms-fresh gyro instead of up-to-16.7 ms-stale." And a steady heartbeat re-transmits current state so a single dropped UDP packet cannot leave a button stuck:

```
apps/controller-ui/src/app/App.tsx · lines 1026-1032 lang
1  useEffect(() => {
2    if (!isActive) return;
3    // Engine runs at a fixed ~60Hz JS heartbeat (native C++ TX thread does the
4    // real 1000Hz send). This re-sends current state so dropped packets recover.
5    const id = setInterval(() => sendTelemetryRef.current(), 16);
6    return () => clearInterval(id);
7  }, [isActive]);
```

The 16 ms interval is the frame budget written as a constant. It embodies the perceptual principle that *convergence to truth must happen at least once per visible frame*, so any transient error is corrected before a human could see it.

**NOTE**

The architecture doc (`ARCHITECTURE.md` §4.5) still says `sendGamepadTelemetry` "allocates a fresh 20-byte `ArrayBuffer`" and applies gyro "additively to the left stick." The shipped code disagrees on both counts — it reuses one `packetRef` buffer and injects gyro in normalized space *after* the deadzone — and where they conflict, the code wins. Both changes are latency wins: no per-packet allocation, no double deadzone.

## Haptics and multimodal feedback

A physical Xbox pad gives you feedback you never think about: the click of a button, the resistance of a trigger, the rumble of an explosion. A sheet of glass gives you none of that. This is why haptics is not a garnish on a touch controller — it is the *replacement* for the missing mechanical channel.

**DEFINITION**

**Multimodal feedback** — Delivering the same event through more than one sense at once (here: sight + touch) so the brain fuses them into one confident percept. A press that you both *see* light up and *feel* buzz reads as more "real" and more responsive than either alone.

Your first psychophysical problem is that phones are not equal. Some motors can vary their strength (amplitude control); many cheap linear motors are strictly on/off. Feeding a "gentle 40%" command to an on/off motor produces *full* strength — so weak and strong rumble feel identical, and the user's

Intensity slider does nothing. `MainActivity.kt` detects the tier and, when there is no amplitude control, *fakes* amplitude with a duty cycle.

#### DEFINITION

**Pulse-width modulation (PWM)** — Emulating a continuous level with a fast on/off square wave, where the fraction of time spent "on" (the duty cycle) sets the perceived intensity. The motor can only slam fully on or fully off, so you trade *time* for *amplitude*.

```
apps/android-client/app/src/main/java/com/gamepad/client/MainActivity.kt · lines 687-702 Kotlin
1          } else if (amplitude in 1..254) {
2              // No amplitude control (e.g. Moto G84): every custom amplitude silently
3              // collapses to FULL strength, so weak rumble feels as strong as strong
4              // rumble – and under sustained rumble the back-to-back full pulses read
5              // as one continuous buzz. Emulate intensity with a duty-cycle (PWM)
6              // waveform: on-time per ~16ms cycle is proportional to the requested
7              // amplitude, so a low value pulses briefly (gentle) and a high value
8              // stays on longer (strong).
9              val period = 16L
10             val on = (period * amplitude / 255).coerceIn(1, period)
11             val off = (period - on).coerceAtLeast(0)
12             val cycles = (durationMs / period).toInt().coerceIn(1, 8)
13             val timings = LongArray(cycles * 2 + 1) { i ->
14                 if (i == 0) 0L else if (i % 2 == 1) on else off
15             }
16             vibrator.vibrate(android.os.VibrationEffect.createWaveform(timings, -1))
```

`on = period * amplitude / 255` is textbook PWM: a requested strength of 64/255 stays on for a quarter of each 16 ms cycle, and the mechanoreceptors in the fingertip — which integrate rapid pulses into a single felt intensity — perceive that as "gentle." This is you re-deriving amplitude perception from raw duty cycle because the hardware refused to give it to you directly.

Above this sits a **capability-tiered** design straight out of HCI's "graceful degradation" playbook. `getHapticCapabilities` probes the device, and `playHaptic` picks the best available rendering: OEM-tuned `Composition` primitives / predefined `EFFECT_CLICK` and `EFFECT_TICK` on capable phones, an atomic double-pulse waveform on amplitude-less LRA motors (so a press+release "double" cannot be swallowed), and a plain oneshot on old APIs. Crucially, the feedback is **semantic**, not raw: `triggerHaptic` maps a legacy millisecond value to an *intent* — `ms >= 28 ? "ripple" : ms >= 18 ? "buttonPress" : "buttonTap` — so the same physical event always feels the same across every device tier.

The feedback is also **mapped to game meaning**, which is what makes it multimodal rather than noise. `onRumblePacket` receives the left and right motor levels the PC's game emitted, and translates them for the phone:

- Strength is `Math.max(leftMotor, rightMotor) * (rumbleIntensity/100)` — the louder motor wins, scaled by the user's slider.
- Pulse *length* scales with strength (`dur = 20 + (strength/255)*80`, i.e. 20–100 ms), so even amplitude-less phones feel the Intensity slider through duration.



- The stick "hits the rim" fires `playHaptic("tick")`, and a gyro "full lock" fires `triggerHaptic(15)` — synthetic detents that give a rubber-band touch stick the *hard stop* a real one has.

**WATCH OUT**

Haptics competes with itself. The comment notes that on single-effect LRA motors a press CLICK gets "cancelled\_superseded" by the release TICK ~45 ms later, collapsing the double into one buzz. The atomic-waveform workaround exists because the motor is a shared, serial resource — a subtlety invisible until you feel it fail.

## Input ergonomics and embodiment

Ergonomics asks whether the control fits the body; **embodied interaction** asks whether the *mapping* fits how the body already thinks.

**DEFINITION**

**Natural (embodied) mapping** — A control scheme where the physical action mirrors the real-world effect, so no conscious translation is needed — tilt the phone right, the car steers right. The knowledge lives in the hands, not the head.

Your gyro tilt-steering is the purest example. Reading `useGyro`, roll angle becomes steering (`1sXn = clamp(1sXn - gx)` in racing mode) or a 3D look axis in 3D mode — the phone is the wheel. Two ergonomic refinements make it usable by a real body rather than an idealised one. First, `CALIBRATE` captures the current pose as neutral, so a player reclining on a couch can define "straight ahead" at whatever angle is comfortable rather than being forced upright. Second, an exponential response curve —  $0.7 * |t| + 0.3 * |t|^3$  — gives fine control near centre and full authority at the edges, matching how a driver wants a wheel to feel.

The circular deadzone is the ergonomics of *tremor and rest*. A thumb never returns to exactly centre, and no human holds perfectly still; a naive controller would drift.

**DEFINITION**

**Deadzone** — A region of no output around the neutral position, so tiny unintended movement produces zero input. A *circular* deadzone uses radial distance, which is uniform in every direction.

apps/controller-ui/src/app/App.tsx · lines 924-935

lang

```

1  const processStick = (x: number, y: number) => {
2    const distance = Math.sqrt(x * x + y * y);
3    const deadzoneRadius = (8 / 100) * 60; // 8% default deadzone
4    if (distance >= deadzoneRadius && distance > 0) {
5      const factor = (distance - deadzoneRadius) / (60 - deadzoneRadius);
6      return {
7        x: (x / distance) * factor * 60,
8        y: (y / distance) * factor * 60
9      };

```

```
apps/controller-ui/src/app/App.tsx · (cont.)
```

```
lang
```

```
10      }  
11      return { x: 0, y: 0 };  
12    };
```

The **factor** term does more than gate — it *re-maps* the surviving range (8%–100%) back onto a full 0–1, so output eases in from zero at the deadzone edge instead of snapping. [ARCHITECTURE.md](#) §7 gives the exact ergonomic reason: "A square deadzone causes diagonal drift. A circular deadzone provides consistent behavior in all directions." That is a claim about the *body's* experience, and here the code matches the doc.

Embodiment also shows up in multitouch discipline. `useStick` owns a single `pointerId` (`pid.current`) so that a stray second finger cannot yank an active stick back to centre — the code comment: "a stray 2nd finger must not recenter an active stick." A physical pad has one thumb per stick by construction; on glass you have to *enforce* that one-finger-one-control embodiment yourself. Finally, the **custom pad** system (`makeDefaultPad`, FPS/fighter/racing/RPG templates, per-widget `x/y/r`) is ergonomics made user-editable: the player sizes and places each target for their own hands. Larger `r` means an easier target — a direct nod to **Fitts's law**, which says acquisition time falls as targets grow and move closer.

#### EXPERT DETAIL

The gyro value fed to the game is already the output of a native 1€ (OneEuro) filter running at ~200 Hz; `App.tsx` adds *no* extra smoothing to that value, because a second filter would double the phase lag. The rAF glide you see on the indicator bar is cosmetic only. Filtering the *display* but not the *control* is the same ref-then-send philosophy applied to perception: never let the pretty channel slow the responsive one.

## Visual and interaction design

The controller UI is dark, near-black, with translucent neon SVG widgets. That is not only taste — it is **figure-ground** management. A black field lets glowing controls read instantly as "the interactive layer," and it hides a load-bearing trick you'll meet in a moment.

#### DEFINITION

**Figure-ground / visual hierarchy** — The perceptual sorting of a scene into an active "figure" and a passive "ground." Good interface design makes the thing you can touch obviously the figure.

The user-mandated gyro indicator is a small masterclass in perceptual layering. Per your own spec (recorded in [SESSION\\_HANDOFF\\_2026-07-04.md](#)), it is a top-edge, edge-to-edge bar, 6.5 px thick (65% of the old thickness), **no glow**, sitting *behind* the buttons in z-order, its fills driven by `scaleX`:

```
apps/controller-ui/src/app/App.tsx · lines 1250-1265
```

```
lang
```

```
1      {gyroMode === "racing" && (  
2        <div className="w-full overflow-hidden flex relative transition-opacity duration-200"  
3          style={{  
4            // Edge-to-edge like 1.2.9, but 65% of its thickness (10px → 6.5px).  
          }}  
        )  
      )  
    }  
  );  
}
```

```
apps/controller-ui/src/app/App.tsx · (cont.) lang
5         height: "6.5px",
6         background: gyroOn ? "rgba(0,212,255,0.14)" : "rgba(255,255,255,0.12)",
7         opacity: gyroOn ? 1 : 0.7,
8     }>
9     {/* center tick */}
10    <div className="absolute left-1/2 top-0 bottom-0 w-[2px] -translate-x-1/2 bg-white/50 z-10" />
11    <div className="flex-1 overflow-hidden">
12        {/* left half — scales from the RIGHT edge (toward center) */}
13        <div ref={gyroTilt.leftBarRef} className="h-full w-full"
14            style={{ transform: "scaleX(0)", transformOrigin: "right center", willChange: "transform
15        },
16            background: "#00d4ff" }} />
17    </div>
```

Two design ideas are fused here. `scaleX` with `transformOrigin: "right center"` means the left half fills *inward from the edge toward the centre tick* — a mapping so the bar grows the way a steering wheel deflects, and (per the render loop) it is GPU-composited with no layout reflow, so the visual feedback never costs a frame. The **z-order** decision is pure figure-ground: the indicator is ambient telemetry, so it must live *behind* the interactive figure (the buttons). The handoff records that equal `z-0` plus DOM order was *not* enough — an explicit `z-[5]` on the button canvas over the `zIndex:0` indicator layer was required to force the bar behind. Perception did not match the naive code, so the code was bent to perception.

That "no glow" rule has a concrete cause, and it produced a second, invisible design element. Buttons are ~90% transparent, so a glowing bar behind them would *shine through* and read as being on top — destroying the figure-ground order. The fix is an opaque black silhouette painted under every translucent button body:

```
apps/controller-ui/src/app/components/Widgets.tsx · lines 118-125 lang
1     /* Opaque black silhouette under the translucent body — invisible on the
2     pure-black screen, but stops the gyro indicator (z-0, behind the canvas)
3     shining through the ~90%-transparent button fills. */}
4     {isRect ? (
5         <rect x={cx - w! / 2} y={cy - h! / 2} width={w!} height={h!} rx={rx} fill="#000" style={{ pointerEv
6         ents: "none" }} />
7     ) : (
8         <circle cx={cx} cy={cy} r={r} fill="#000" style={{ pointerEvents: "none" }} />
9     )}
```

The `#000` silhouette is invisible against the black screen but opaque against the bar — a purely perceptual object that exists only to protect the figure-ground relationship the dark theme depends on. This is where the dark UI stops being a mood and becomes load-bearing engineering.

#### NOTE

HCI appears only *lightly* in a few corners — there is no formal accessibility layer (colour-blind palettes, text scaling, one-handed reachability presets), and haptic/visual sync is tuned by feel rather than measured against a JND threshold. Those are the natural next chapters if you ever want the perceptual science to become as rigorous as the latency engineering already is.

---

DISCIPLINE 10

# **Software Engineering, Databases & Operations Research**

10

Three disciplines quietly hold GamepadOS together once you look past the physics of the input path. They are less about clever tricks and more about *keeping a growing system understandable, correct, and efficient over time*. This section introduces each from zero and shows exactly where your own code embodies it.

**DEFINITION**

**Software Engineering** — the study of how to build and maintain software as a durable *system* — how to divide it into parts, how those parts should talk to each other, and how to change it safely over months and years without breaking what works.

**DEFINITION**

**Database Theory** — the mathematics of storing facts without contradiction. Its relational branch models data as *tables of rows* and uses set theory and logic to guarantee that certain nonsense states (two rows that should be one, a child pointing at a missing parent) simply cannot exist.

**DEFINITION**

**Operations Research (OR)** — the applied-mathematics discipline of making the *best decision* under constraints — routing, scheduling, assigning limited resources — usually by defining a quantity to minimise or maximise and searching for the option that does it.

You will meet all three below. Software architecture and process are everywhere in GamepadOS; database theory lives almost entirely in the support website; operations research appears in exactly one small, elegant place — the ticket auto-router — and nowhere else, which is itself worth saying honestly.

## Software Architecture — Separation of Concerns Across Layers

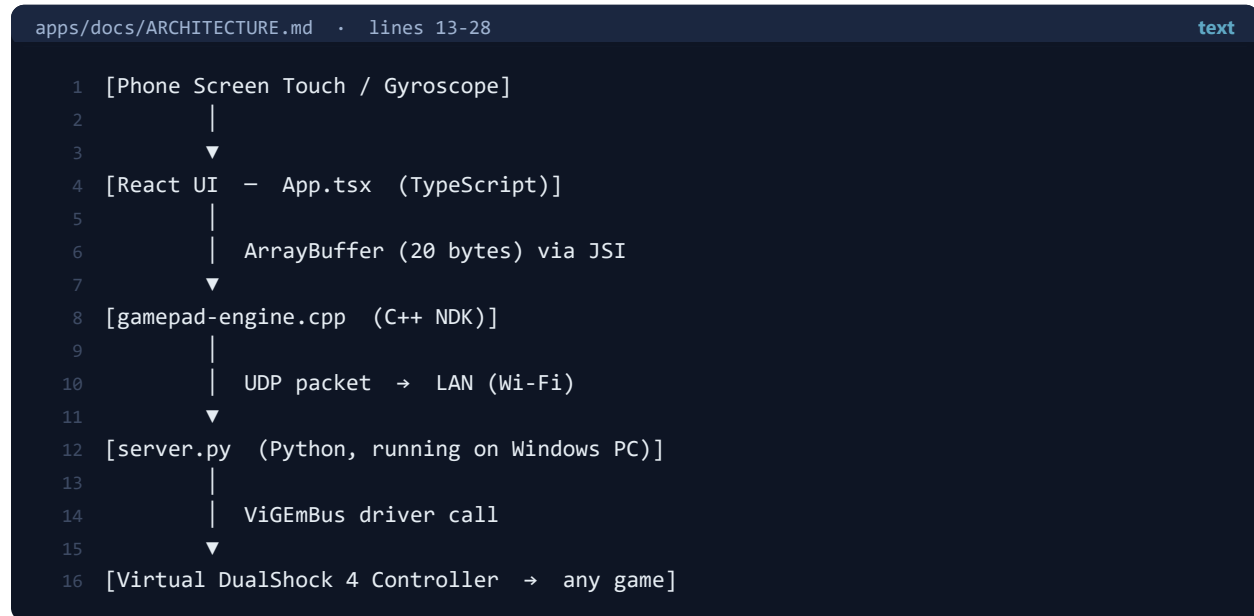
**DEFINITION**

**Software Architecture** — the high-level shape of a system — what the major pieces are and where the boundaries between them sit — chosen *before* you write most of the code because it is the most expensive thing to change later.

**DEFINITION**

**Separation of Concerns** — the principle that each part of a system should be responsible for one kind of job, so you can reason about (and change) that job without holding the whole system in your head.

GamepadOS is a textbook example of separation of concerns applied across a vertical stack. Your own architecture document draws the boundary lines explicitly.



Each box is a *concern* with a single job: the React UI concerns itself only with what a human sees and touches; the C++ engine concerns itself only with getting bytes onto the wire with minimal jitter; the Python server concerns itself only with turning bytes into a virtual controller. The arrows are the important part — they are the *interfaces*, and they are deliberately narrow. Between UI and engine the entire contract is a 20-byte **ArrayBuffer**; between engine and server it is a 20-byte UDP packet. A narrow interface is what lets you rewrite one box without touching the others.

**DEFINITION**

**Interface (in architecture)** — the agreed contract between two components — the only thing each side is allowed to assume about the other. A good interface hides everything else.

**NOTE**

The 20-byte packet is the single most valuable architectural decision in the project. Because the contract is *bytes*, not function calls, the language on each side is irrelevant. Your Swift/iOS port can replace the entire Kotlin+C++ column and still speak to the unchanged Python server, because it emits the same 20 bytes. Architecture measured well: one whole layer is swappable.

## Coupling, Cohesion, and the Deliberate **App.tsx** Monolith

**DEFINITION**

**Coupling** — how much two components depend on each other's internals. Low coupling is good — you can change one without breaking the other.

**DEFINITION**

**Cohesion** — how strongly the things *inside* one component belong together. High cohesion is good — the file is about one thing.

The interesting tension in GamepadOS is *inside* the React layer. `App.tsx` is a single file of 4,261 lines holding every screen, all state, all rendering, and all packet construction. Conventional advice says "split large files up." You did the opposite, on purpose, and your architecture doc defends it.

apps/docs/ARCHITECTURE.md · lines 273-273

text

```
1 | **Single `App.tsx` monolith** | All controller state is deeply interconnected. Splitting into files creat
   | ed prop-drilling nightmares. Colocating everything makes the data flow obvious. |
```

This is a genuine engineering judgement, not laziness. In a controller, the polling loop, the gyro tilt, the deadzone, the held-button set, and the custom-pad layout are all *one tightly-coupled concern* — they must agree, 60 times a second, to build one correct packet. Splitting them into modules would not reduce coupling; it would only *hide* it behind React props passed down through layers ("prop-drilling"), turning a visible dependency into an invisible one. The monolith is a case where high internal cohesion (everything here really is about "produce the next controller frame") justifies a large file. The cost is real — merge conflicts, slow editor, hard onboarding — and the honest reading is: the *right* boundary here is the 20-byte interface at the bottom of the file, not arbitrary line-count-driven splits inside it.

#### WATCH OUT

ARCHITECTURE.md line 103 calls `App.tsx` "~3800 lines." The file is now 4,261 lines. When a doc and the code disagree, the code wins — the doc is simply stale. Treat the *shape* of the claim (a large deliberate monolith) as true and the number as approximate.

## The Hybrid-App Pattern, Reused

#### DEFINITION

**Hybrid App** — an app whose user interface is a web page (HTML/JS/CSS) running inside a native "shell," with a small bridge letting the web code call native hardware features. You write the UI once and reuse it across native platforms.

Your Android client is a hybrid app: a thin Kotlin `WebView` hosts the compiled React bundle, and a `JavaScriptInterface` bridges into C++. The architectural payoff is that this *pattern* — not just the code — is reusable. The iOS port applies the identical shape: a Swift `WKWebView` hosts the *same* `controller-ui` bundle, with a CryptoKit/Swift engine behind an analogous bridge. The concern boundaries (UI ↔ shell ↔ engine) are platform-independent, so porting means re-implementing two boxes, not the whole system. That is architecture paying rent: a good decomposition made a second platform a bounded job instead of a rewrite.

## Relational Database Theory — Models as Tables

Everything above was structure in *code*. Relational database theory is structure in *stored facts*, and it lives in your support website's `schema.prisma`.

**DEFINITION**

**Relation (table)** — in the relational model, a set of rows that all share the same columns. "Set" is literal — order does not matter and duplicate rows are not allowed. Each Prisma `model` is one relation.

**DEFINITION**

**Primary Key** — a column whose value uniquely identifies a row. No two rows may share it; it is the row's identity.

**DEFINITION**

**Foreign Key** — a column in one table holding the primary key of a row in another table — a typed pointer that ties the two tables together.

website/backend/prisma/schema.prisma · lines 37-50

Prisma schema

```

1  model TicketMessage {
2    id      String   @id @default(uuid())
3    ticketId String
4    ticket  Ticket   @relation(fields: [ticketId], references: [id], onDelete: Cascade)
5    sender  String   // 'user' | 'admin'
6    via     String   @default("portal") // 'portal' (admin dashboard) | 'email' (inbound email reply)
7    authorName String? // which admin wrote it (null for user messages)
8    internal Boolean @default(false) // internal note: admins only, never emailed to the user
9    body    String
10   attachments String? // JSON string array of {url, name, size}
11   createdAt DateTime @default(now())
12
13   @@index([ticketId])
14 }

```

This one model shows the whole relational vocabulary. `@id` marks `id` as the **primary key** — a UUID, so identity never collides. `ticketId` is a **foreign key**: `@relation(fields: [ticketId], references: [id])` says "this column points at a `Ticket`'s `id`." The pair `Ticket 1 — many TicketMessage` is a **one-to-many relationship**, the backbone of the whole ticketing model: one conversation, many messages. `onDelete: Cascade` encodes a rule of *referential integrity* — delete a ticket and its messages vanish with it, so you can never have an orphaned message pointing at a ticket that no longer exists.

**DEFINITION**

**Referential Integrity** — the guarantee that every foreign key points at a row that actually exists. Databases enforce it so your data can never contradict itself.

**DEFINITION**

**Index** — a secondary lookup structure (like a book's index) that makes "find all rows where X" fast without scanning the whole table. `@@index([ticketId])` builds one so loading a ticket's messages stays quick as the table grows.

## Normalization and the Set-Theoretic Uniqueness Invariant



**DEFINITION**

**Normalization** — organising tables so each fact is stored in exactly one place. The goal is to eliminate redundancy, because a fact stored twice is a fact that can disagree with itself.

Your schema is largely normalized: a ticket's replies live only in `TicketMessage`, an admin's identity lives only in `Admin`, and other tables *reference* them by key rather than copying them. There is one deliberate, documented exception, and naming it shows you understand the rule you are bending.

website/backend/prisma/schema.prisma · lines 108-119

Prisma schema

```

1 model AuditLog {
2   id      String   @id @default(uuid())
3   adminId String?
4   admin   Admin?   @relation(fields: [adminId], references: [id], onDelete: SetNull)
5   adminName String  // denormalized so entries survive admin removal
6   action  String   // e.g. 'login', 'reply', 'status', 'assign', 'admin.create'
7   ticketId String?
8   detail  String?
9   createdAt DateTime @default(now())
10
11   @@index([createdAt])
12 }

```

`adminName` is stored even though it could be looked up through `adminId` — that is **denormalization**, chosen on purpose. The comment says why: an audit log must remain readable *after* the admin is deleted (`onDelete: SetNull` blanks the foreign key). History that changes when you delete a person is not history. So you copy the name at write-time to freeze it. Denormalization done knowingly, for an audit trail, is correct; the discipline is in the comment that says "I know this is redundant, and here is the reason."

The sharpest piece of database theory in the whole project is the download counter's uniqueness constraint.

website/backend/prisma/schema.prisma · lines 63-73

Prisma schema

```

1 model Download {
2   id      String   @id @default(uuid())
3   asset   String   // 'pc' (GamepadServer-Setup.exe) | 'app' (GamepadOS.apk)
4   ipHash  String   // salted SHA-256 of the client IP (privacy-preserving)
5   day     String   // 'YYYY-MM-DD' (UTC) — the per-day dedupe window
6   kind    String   @default("install") // 'install' = fresh download (website button) | 'update' = in-app
7   update check
8   createdAt DateTime @default(now())
9
10   @@unique([asset, ipHash, day, kind])
11   @@index([asset])
12 }

```

**DEFINITION**

**Composite Unique Constraint** — a rule that a *combination* of columns must be unique across the whole table, even if each column individually repeats. It defines a compound key: the tuple, not any single field, is the identity.

`@@unique([asset, ipHash, day, kind])` is a **set-theoretic invariant**. It declares that the tuple `(asset, ipHash, day, kind)` is a *key* — at most one row can exist for "this app, this hashed IP, this UTC day, this event kind." The database itself becomes the deduplicator: you don't check-then-insert (which races), you just insert and let the constraint reject the duplicate. The insert code leans on exactly this.

```
website/backend/server.js · lines 1150-1157 JavaScript
1  async function recordDownload(asset, ip, kind = 'install') {
2    try {
3      await prisma.download.create({ data: { asset, ipHash: ipHashOf(ip), day: todayUTC(), kind } });
4    } catch (e) {
5      if (!(e && e.code === 'P2002')) console.warn('recordDownload error:', e.message);
6      return; // P2002 = same person/day already counted → no new unique → no broadcast
7    }
8    try { sseBroadcast('download:update', await downloadCounts()); } catch {}
```

Prisma error `P2002` is "unique constraint violated." The code treats it not as a failure but as *information*: "this person already counted today, so there is no new unique user — return silently." The headline count is then just the number of distinct rows, computed by `groupBy(['ipHash'])`. This is the relational model working as designed — a mathematical guarantee (`@@unique`) enforced by the engine, turned into a correct, race-free, deduplicated counter without a single manual "have I seen this IP?" lookup. And `ipHashOf` salts and SHA-256s the IP so the invariant is preserved *without* storing anyone's real address — correctness and privacy from one design.

#### IN PLAIN TERMS

Imagine a nightclub with a hand-stamp. When you enter, the bouncer stamps your hand with today's date. Try to get counted twice today and the stamp's already there — you're one visitor, not two.

`@@unique([asset, ipHash, day, kind])` is that stamp, and `P2002` is the bouncer saying "already stamped, move along." The club never needs a list of who's inside; the stamps *are* the count.

## Software-Engineering Process — Versioning, Signing, Rollback

#### DEFINITION

**Software Process** — the disciplined, repeatable procedures around the code — how you version it, ship it, verify it, and undo it — as opposed to the code itself. Bugs escape through weak process far more often than through weak code.

#### DEFINITION

**Semantic Versioning** — a naming scheme `MAJOR.MINOR.PATCH` (e.g. `1.3.0`) where the numbers communicate the *size* of a change to humans and machines: patch = fix, minor = feature, major = breaking.

Your `RELEASE.md` is a process artifact — a written procedure so a release is repeatable and not reinvented (or botched) each time. It encodes several distinct engineering principles at once.

RELEASE.md · lines 70-72

text

```

1 `apps/android-client/app/build.gradle.kts`:
2 - `versionCode = 11` ← **increment the integer every release** (Android compares this; the in-app updater
   only offers an update when the server's versionCode is higher)
3 - `versionName = "1.1.9"` ← human-readable

```

**DEFINITION**

**Monotonicity** — a quantity that only ever moves in one direction. `versionCode` must be *monotonically increasing* — strictly larger every release.

Two separate ideas sit here. `versionName` ("1.3.0") is **semantic versioning for humans**. `versionCode` (an integer, now 22) is **monotonicity for machines**: the updater offers an update *only* when the server's code is strictly higher, so a version that never goes backwards is what makes "is there an update?" a safe, total ordering rather than a guess. Break monotonicity and the updater either never fires or fires forever — which your doc calls out in its opening lines.

**DEFINITION**

**Code Signing** — cryptographically stamping a build with a private key so the installer can verify it came from you and refuse impostors.

**DEFINITION**

**Regression** — a change that breaks something that used to work. Guarding against regressions is a core reason process exists.

RELEASE.md · lines 89-90

text

```

1 > **Must be signed with the same `release.keystore`** as every prior release. The
2 > in-app updater hands the APK to the system installer, which **rejects an update

```

Signing every APK with the *same* keystore is what makes an update an update and not a stranger's app — Android refuses to replace a signed app with one signed by a different key. And your "unique filename, never overwrite" rule (`GamepadOS-1.1.9.apk`, not `GamepadOS.apk`) is a **rollback/archive discipline**: keeping every prior build means a bad release can be undone by re-activating the previous one, because it still physically exists.

Finally, the doc enforces **single source of truth**: it marks `server.py`'s `APP_VERSION` as "the one that decides 'is an update available'." When one fact (the running version) has one authoritative home, the whole updater has a single point of correctness rather than several copies that can drift — the same anti-redundancy instinct as database normalization, applied to source code.

**NOTE**

Automated testing is the one process pillar that appears only lightly. `RELEASE.md` step 6 defines a manual end-to-end test ("open a server on the old version → update → confirm it lands on the new one") as "the only true end-to-end test of the updater." That is a real, disciplined *manual* gate; there is no automated regression suite behind it. For a solo project this is a defensible trade — but it is the thinnest link in an otherwise strong process chain, and worth naming.

## Operations Research — The Ticket Router as an Assignment Problem

Now the one place OR genuinely appears. When a support ticket arrives, someone must own it. Handing every ticket to the same admin overloads them; handing them out at random is unfair. This is a classic OR problem.

**DEFINITION**

**Assignment / Load-Balancing Problem** — given tasks and workers, decide who does what so that work is spread evenly (or some cost is minimised). It is one of the foundational problems of operations research.

**DEFINITION**

**Objective Function** — the single number an optimisation tries to make as small (or large) as possible. Choosing what to minimise is the modelling.

Your `pickAssignee` solves a small instance of exactly this.

website/backend/server.js · lines 303-327

JavaScript

```
1  async function pickAssignee(subject) {
2    try {
3      // Auto-routing is ALWAYS on – every ticket gets an owner/admin and is never
4      // silently left unassigned. (The portal no longer exposes an off switch.)
5      const ownersOnly = subject === 'business';
6      let candidates = await prisma.admin.findMany({
7        where: { active: true, ...(ownersOnly ? { role: 'owner' } : {}) },
8        orderBy: { createdAt: 'asc' },
9      });
10     // Safety net: a business ticket with no active owner falls back to anyone.
11     if (!candidates.length && ownersOnly) {
12       candidates = await prisma.admin.findMany({ where: { active: true }, orderBy: { createdAt: 'asc' } });
13     }
14     if (!candidates.length) return null;
15     const scored = await Promise.all(candidates.map(async (c) => ({
16       admin: c,
17       load: await prisma.ticket.count({ where: { assigneeId: c.id, status: { not: 'resolved' } } }),
18     })));
19     scored.sort((a, b) => a.load - b.load); // candidates already ordered by seniority → stable tie-break
20     return scored[0].admin;
21   } catch (e) {
22     console.warn('Auto-assign failed, leaving ticket unassigned:', e.message);
23     return null;
24   }
25 }
```

website/backend/server.js · (cont.)

JavaScript

```

24   }
25   }

```

Read this as an optimisation over a finite set. The **feasible set** is `candidates` — the active admins (narrowed to owners when `subject === 'business'`, a hard constraint). The **objective function** is `load`, defined as each admin's count of non-resolved tickets. The **optimiser** is `sort((a,b) => a.load - b.load)` picking `scored[0]`: the admin with the minimum current load. That is *greedy assignment to the least-loaded worker* — the standard heuristic for online load-balancing, where tickets arrive one at a time and each must be placed without knowing the future.

Two OR refinements are worth flagging. First, **constraint handling with a fallback**: business tickets are restricted to owners, but if no active owner exists the code widens the feasible set to everyone rather than failing — a soft constraint that degrades gracefully. Second, **deterministic tie-breaking**: candidates are pre-ordered `createdAt: 'asc'` (seniority) and the sort is stable, so equal-load admins always resolve the same way. Determinism matters — a router that shuffles ties makes bugs unreproducible.

| OR concept            | Where in <code>pickAssignee</code>                                |
|-----------------------|-------------------------------------------------------------------|
| Feasible set          | <code>candidates</code> (active admins; owners only for business) |
| Objective (minimise)  | <code>load</code> = count of non-resolved owned tickets           |
| Optimiser             | <code>sort(a.load - b.load) → scored[0]</code>                    |
| Constraint + fallback | <code>ownersOnly</code> , widening if no owner                    |
| Tie-break             | stable sort over <code>createdAt: 'asc'</code>                    |

**EXPERT DETAIL**

This is greedy *online* load-balancing, not the globally optimal *offline* assignment (which, for minimising the maximum load, is the makespan-minimisation problem — NP-hard in general and classically solved by algorithms like Hungarian matching or LPT scheduling). Greedy-least-loaded is the pragmatic online counterpart:  $O(n)$  per arrival, provably within a small constant factor of optimal for makespan, and it never needs to see future tickets. For a support queue of a handful of admins, the gap between this and an optimal solver is a rounding error — and re-running on every arrival (line 543-550) means the system self-corrects as tickets resolve. Choosing the cheap, good-enough algorithm over the expensive optimal one is itself an operations-research decision.

The router is the whole of OR in GamepadOS, and that is fine to say plainly: the input path, the crypto, and the rendering are governed by other disciplines. Operations research shows up once, in a dozen lines, and does its one job — spreading human workload fairly — exactly right.